

MOD-3

How DevOps Affects Architecture

Software architecture is a vast subject, and in this book we will focus on the aspects of architecture that have the largest effects on **Continuous Delivery (CD)** and DevOps, and vice versa.

In this chapter, we will look at:

- Aspects of software architecture
- Basic terminology and goals
- Anti-patterns, such as the monolith
- The fundamental principle of the separation of concerns
- Three-tier systems and microservices

We will finally conclude with some practical issues regarding database migration.

It's quite a handful, so let's get started!

Introducing software architecture

We will discuss how DevOps affects the architecture of our applications rather than the architecture of software deployment systems, which we discuss elsewhere in the book.

Often while discussing software architecture, we think of the non-functional requirements of our software. By non-functional requirements, we mean the different characteristics of the software rather than the requirements of particular behaviors.

A functional requirement could be that our system should be able to deal with credit card transactions. A non-functional requirement could be that the system should be able to manage several such credit card transactions per second.

Here are two of the non-functional requirements that DevOps and CD place on software architecture:

- We need to be able to deploy small changes often
- We need to be able to have great confidence in the quality of our changes

The normal case should be that we are able to deploy small changes all the way from developers' machines to production in a small amount of time. Rolling back a change because of unexpected problems caused by it should be a rare occurrence.

So, if we take out the deployment systems from the equation for a while, how will the architecture of the software systems we deploy be affected?

The monolithic scenario

One way to understand the issues that a problematic architecture can cause for CD is to consider a counter-example for a while.

Let's suppose we have a large web application with many different functions. We also have a static website inside the application. The entire web application is deployed as a single Java EE application archive. So, when we need to fix a spelling mistake in the static website, we need to rebuild the entire web application archive and deploy it again.

While this may be seen as a silly example, and the enlightened reader would never do such a thing, I have seen this anti-pattern live in the real world. As DevOps engineers, this could be an actual situation that we may be asked to solve.

Let's break down the consequences of this tangling of concerns. What happens when we want to correct a spelling mistake? Let's take a look:

1. We know which spelling mistake we want to correct, but in which code base do we need to do it? Since we have a monolith, we need to make a branch in our code base's revision control system. This new branch corresponds to the code that we have running in production.
2. Make the branch and correct the spelling mistake.
3. Build a new artifact with the correction. Give it a new version.
4. Deploy the new artifact to production.

Ok, this doesn't seem altogether too bad at first glance. But consider the following:

- We made a change to the monolith that our entire business critical system comprises. If something breaks while we are deploying the new version, we lose revenue by the minute. Are we really sure that the change affects nothing else?
- It turns out that we didn't really just limit the change to correcting a spelling mistake. We also changed a number of version strings when we produced the new artifact. But changing a version string should be safe too, right? Are we sure?

The point here is that we have already spent considerable mental energy in making sure that the change is really safe. The system is so complex that it becomes difficult to think about the effects of changes, even though they may be trivial.

Now, a change is usually more complex than a simple spelling correction. Thus, we need to exercise all aspects of the deployment chain, including manual verification, for all changes to a monolith.

We are now in a place that we would rather not be.

Architecture rules of thumb

There are a number of architecture rules that may help us understand how to deal with the previous undesirable situation.

The separation of concerns

The renowned Dutch computer scientist, Edsger Dijkstra, first mentioned his idea of how to organize thought efficiently in his paper from 1974, *On the role of scientific thought*.

He called this idea *the separation of concerns*. To this date, it is arguably the single most important rule in software design. There are many other well-known rules, but many of them follow from the idea of the separation of concerns. The fundamental principle is simply that we should consider different aspects of a system separately.

The principle of cohesion

In computer science, *cohesion* refers to the degree to which the elements of a software module belong together.

Cohesion can be used as a measure of how strongly related the functions in a module are. It is desirable to have strong cohesion in a module.

We can see that strong cohesion is another aspect of the principle of the separation of concerns.

Coupling

Coupling refers to the degree of dependency between two modules. We always want low coupling between modules.

Again, we can see coupling as another aspect of the principle of the separation of concerns.

Systems with high cohesion and low coupling would automatically have separation of concerns, and vice versa.

Back to the monolithic scenario

In the previous scenario with the spelling correction, it is clear that we failed with respect to the separation of concerns. We didn't have any modularization at all, at least from a deployment point of view. The system appears to have the undesirable features of low cohesion and high coupling.

If we had a set of separate deployment modules instead, our spelling correction would most likely have affected only a single module. It would have been more apparent that deploying the change was safe.

How this should be accomplished in practice varies, of course. In this particular example, the spelling corrections probably belong to a frontend web component. At the very least, this

frontend component can be deployed separately from the backend components and have their own life cycle.

In the real world, though, we may not be lucky enough to always be able to influence the different technologies used by the organization where we work. The frontend may, for instance, be implemented using a proprietary content management system with quirks of its own. Where you experience such circumstances, it would be wise to keep track of the cost such a system causes.

A practical example

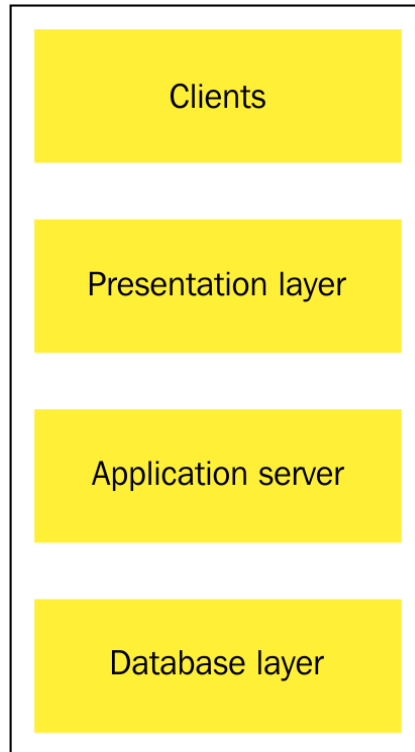
Let's now take a look at the concrete example we will be working on for the remainder of the book. In our example, we work for an organization called Matangle. This organization is a **software as a service (SaaS)** provider that sells access to educational games for school children. As with any such provider, we will, with all likelihood, have a database containing customer information. It is this database that we will start out with.

The organization's other systems will be fleshed out as we go along, but this initial system serves well for our purposes.

Three-tier systems

The Matangle customer database is a very typical three-tier, **CRUD (create, read, update, and delete)** type of system. This software architecture pattern has been in use for decades and continues to be popular. These types of systems are very common, and it is quite likely that you will encounter them, either as legacy systems or as new designs.

In the following diagram, we can see the separation of concerns idea in action:



The three tiers listed as follows show examples of how our organization has chosen to build its system.

The presentation tier

The presentation tier will be a web whose frontend is implemented using the React web framework. It will be deployed as a set of JavaScript and static HTML files. The React framework is fairly recent and your organization may not use React but perhaps some other framework, such as Angular, instead. In any case, from a deployment and build point of view, most JavaScript frameworks are similar.

The logic tier

The logic tier is a backend which is implemented using the Clojure language on the Java platform. The Java platform is very common in large organizations, while smaller organizations may prefer other platforms based on Ruby or Python. Our example, based on Clojure, contains a little bit of both worlds.

The data tier

In our case, the database is implemented with the PostgreSQL database system. PostgreSQL is a relational database management system. While arguably not as common as MySQL installations, larger enterprises may prefer Oracle databases. PostgreSQL is, in any case, a robust system, and our example organization has chosen PostgreSQL for this reason.

From a DevOps point of view, the three-tier pattern looks compelling, at least superficially. It should be possible to deploy changes to each of the three layers separately, which would make it simple to propagate small changes through the servers.

In practice, though, the data tier and logic tier are often tightly coupled. The same may also be true for the presentation tier and logic tier. To avoid this, care must be taken to keep the interfaces between the tiers lean. Using well-known patterns isn't necessarily a panacea. If we don't take care while designing our system, we can still wind up with an undesirable monolithic system.

Handling database migrations

Handling changes in a relational database requires special consideration.

A relational database stores both data and the structure of the data. Upgrading a database schema offers other challenges than the ones present when upgrading program binaries. Usually, when

we upgrade an application binary (A binary file is a file whose content must be interpreted by a program or a hardware processor that understands in advance exactly how it is formatted. Programmers often talk about an executable program as a "binary" and will ask another programmer to "send me the binaries". A synonym for this usage is object code), we stop the application, upgrade it, and then start it again. We don't really bother about the application state. That is handled outside of the application.

When upgrading databases, we do need to consider state, because a database contains comparatively little logic and structure, but contains much state.

To describe a database structure change, we issue a command to change the structure.

The database structures before and after a change is applied should be seen as different versions of the database. How do we keep track of database versions?

Liquibase is a database migration system that, at its core, uses a tried and tested method. There are many systems like this, usually at least one for every programming language. Liquibase is well-known in the Java world, and even in the Java world, there are several alternatives that work in a similar manner. Flyway is another example for the Java platform.

Generally, database migration systems employ some variant of the following method:

1. Add a table to the database where a database version is stored.
2. Keep track of database change commands and bunch them together in versioned change sets. In the case of Liquibase, these changes are stored in XML files. Flyway employs a slightly different method where the change sets are handled as separate SQL files or occasionally as separate Java classes for more complex transitions.
3. When Liquibase is being asked to upgrade a database, it looks at the metadata table and determines which change sets to run to make the database up-to-date with the latest version.

As previously stated, many database version-management systems work like this. They differ mostly in the way the change sets are stored and how they determine which change sets to run. They may be stored in an XML file, as in the case of Liquibase (Liquibase supports several formats, such as XML, YAML, JSON, and SQL), or as a set of separate SQL files, as with

Flyway. This later method is more common with homegrown systems and has some advantages. The Clojure ecosystem also has at least one similar database migration system of its own, called Migratus.

The following table highlights a handful of example schema migration tools:

Language	Name	Site
Java	Liquibase	http://www.liquibase.org/
Java	Flyway	https://flywaydb.org/
Clojure	Migratus	https://github.com/yogthos/migratus
Python	Alembic	https://pypi.python.org/pypi/alembic

Rolling upgrades

Another thing to consider when doing database migrations is what can be referred to as rolling upgrades. These kinds of deployments are common when you don't want your end user to experience any downtime, or at least very little downtime.

Here is an example of a rolling upgrade for our organization's customer database.

When we start, we have a running system with one database and two servers. We have a load balancer in front of the two servers.

We are going to roll out a change to the database schema, which also affects the servers. We are going to split the customer name field in the database into two separate fields: first name and surname.

This is an incompatible change. How do we minimize downtime? Let's look at the solution:

1. We start by doing a database migration that creates the two new name fields and then fills these new fields by taking the old name field and splitting the field into two halves by finding a space character in the name. This was the initial chosen encoding for names, which wasn't stellar. This is why we want to change it.
2. This change is so far backward compatible, because we didn't remove the name field; we just created two new fields that are, so far, unused.
3. Next, we change the load balancer configuration so that the second of our two servers is no longer accessible from the outside world. The first server chugs along happily, because the old name field is still accessible to the old server code.
4. Now we are free to upgrade server two, since nobody uses it.
5. After the upgrade, we start it, and it is also happy because it uses the two new database fields.
6. At this point, we can again switch the load balancer configuration such that server one is not available, and server two is brought online instead. We do the same kind of upgrade on server one while it is offline. We start it and now make both servers accessible again by re-instating our original load balancer configuration.

Now the change is deployed almost completely. The only thing remaining is removing the old name field, since no server code uses it anymore.

As we can see, rolling upgrades require a lot of work in advance to function properly. It is far easier to schedule upgrades during natural downtimes, if your organization has any. International organizations may not have any suitable natural windows to perform upgrades, so rolling upgrades may be the only option.

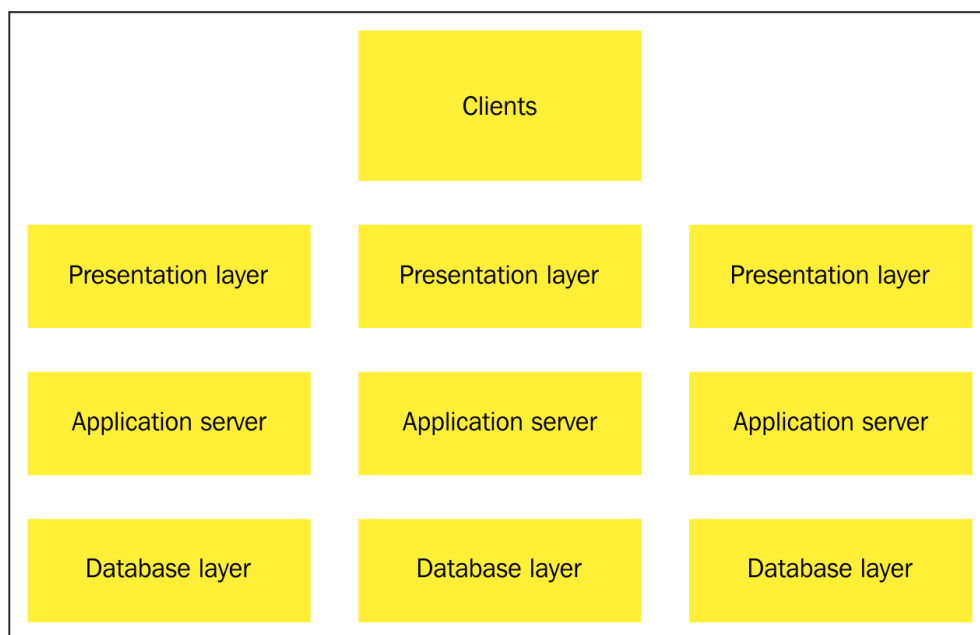
Microservices

Microservices is a recent term used to describe systems where the logic tier of the three-tier pattern is composed of several smaller services that communicate with language-agnostic protocols.

Typically, the language-agnostic protocol is HTTP based, commonly JSON REST, but this is not mandatory. There are several possibilities for the protocol layer.

This architectural design pattern lends itself well to a CD approach since, as we have seen, it's easier to deploy a set of smaller standalone services than a monolith.

Here is an illustration of what a microservices deployment may look like:



We will evolve our example towards the microservices architecture as we go along.

Interlude – Conway's law

In 1968, Melvin Conway introduced the idea that the structure of an organization that designs software winds up copied in the organization of the software. This is called **Conway's law**.

The three-tier pattern, for instance, mirrors the way many organizations' IT departments are structured:

- The database administrator team, or DBA team for short

- The backend developer team
- The frontend developer team
- The operations team

Well, that makes four teams, but we can see the resemblance clearly between the architectural pattern and the organization.

The primary goal of DevOps is to bring different roles together, preferably in cross-functional teams. If Conway's law holds true, the organization of such teams would be mirrored in their designs.

The microservice pattern happens to mirror a cross-functional team quite closely.

Microservices and the data tier

One way of viewing microservices is that each microservice is potentially a separate three-tier system. We don't normally implement each tier for each microservice, though. With this in mind, we see that each microservice can implement its own data layer. The benefit would be a potential increase of separation between services.

It is more common in my experience, though, to put all of the organization's data into a single database or at least a single database type. This is more common, but not necessarily better.

There are pros and cons to both scenarios. It is easier to deploy changes when the systems are clearly separate from each other. On the other hand, data modeling is easier when everything is stored in the same database.

DevOps, architecture, and resilience

We have seen that the microservice architecture has many desirable properties from a DevOps point of view. An important goal of DevOps is to place new features in the hands of our user faster. This is a consequence of the greater amount of modularization that microservices provide. Those who fear that microservices could make life uninteresting by offering a perfect solution without drawbacks can take a sigh of relief, though. Microservices do offer challenges of their own.

We want to be able to deploy new code quickly, but we also want our software to be **reliable**.

Microservices have more **integration points between systems** and **suffer from a higher possibility of failure** than monolithic systems.

Automated testing is very important with DevOps so that the changes we deploy are of good quality and can be relied upon. This is, however, not a solution to the problem of services that suddenly stop working for other reasons. Since we have **more running services** with the microservice pattern, it is **statistically more likely for a service to fail**.

We can partially mitigate this problem by making an effort to monitor the services and take appropriate action when something fails. This should preferably be automated.

In our customer database example, we can employ the following strategy:

1. We use two application servers that both run our application
2. The application offers a special monitoring interface via JsonRest
3. A monitoring daemon periodically polls this monitoring interface

4. If a server stops working, the load balancer is reconfigured such that the offending server is taken out of the server pool

This is obviously a simple example, but it serves to illustrate the challenges we face when designing resilient systems that comprise many moving parts and how they may affect architectural decisions.

Why do we offer our own application-specific monitoring interface, though? Since the purpose of monitoring is to give us a thorough understanding of the current health of the system, we normally monitor many aspects of the application stack. We monitor that the server CPU isn't overloaded, that there is sufficient disk and memory space available, and that the base application server is running. This may not be sufficient to determine whether a service is running properly, though. For instance, the service may for some reason have a broken database access configuration. A service-specific health probing interface would attempt to contact the database and return the status of the connection in the return structure.

It is, of course, best if your organization can agree on a common format for the probing return structure. The structure will also depend on the type of monitoring software used.

Everything is Code

Everything is code, and you need somewhere to store it. The organization's source code management system is that place.

Developers and operations personnel share the same central storage for their different types of code.

There are many ways of hosting the central code repository:

- You can use a piece of software as a service solution, such as GitHub, Bitbucket, or GitLab. This can be cost-effective and provide good availability.
- You can use a cloud provider, such as AWS or Rackspace, to host your repositories.

Some types of organization simply can't let their code leave the building. For them, a private in-house setup is the best choice.

In this chapter, we will explore different options, such as Git, and web-based frontends to Git, such as Gerrit and GitLab.

This exploration will serve to help you find a Git-hosting solution that covers your organization's needs.

In this chapter, we will start to experience one of the challenges in the field of DevOps—there are so many solutions to choose from and explore! This is particularly true for the world of source code management, which is central to the DevOps world.

Therefore, we will also introduce the software virtualization tool, Docker, from a user's perspective, so that we can use the tool in our exploration.

The need for source code control

Terence McKenna, an American author, once said that everything is code.

While one might not agree with McKenna about whether everything in the universe can be represented as code, for DevOps purposes, indeed nearly everything can be expressed in codified form, including the following:

- The applications that we build
- The infrastructure that hosts our applications
- The documentation that documents our products

Even the hardware that runs our applications can be expressed in the form of software.

Given the importance of code, it is only natural that the location where we place code, the source code repository, is central to our organization. Nearly everything we produce travels through the code repository at some point in its life cycle.

The history of source code management

In order to understand the central need for source code control, it can be illuminating to have a brief look at the development history of source code management. This gives us an insight into the features that we ourselves might need. Some examples are as follows:

- **Storing historical versions of source in separate archives:** This is the simplest form, and it still lives on to some degree, with many free software projects offering tar archives of older releases to download.
- **Centralized source code management with check in and check out:** In some systems, a developer can lock files for exclusive use. Every file is managed separately. Tools like this include **Revision Control System (RCS)** and **Source Code Control System (SCCS)**.

Normally, you don't encounter this class of tool anymore, except the occasional file header indicating that a file was once managed by RCS.

- **A centralized store where you merge before you commit:** Examples include **Concurrent Versions System (CVS)** and **Subversion**. Subversion in particular is still in heavy use. Many organizations have centralized workflows, and Subversion implements such workflows well enough for them.
- **A decentralized store:** On each step of the evolutionary ladder, we are offered more flexibility and concurrency as well as faster and more efficient workflows. We are

also offered more advanced and powerful guns to shoot ourselves in the foot with, which we need to keep in mind!

Currently, Git is the most popular tool in this class, but there are many other similar tools in use, such as Bazaar and Mercurial.

Time will tell whether Git and its underlying data model will fend off the contenders to the throne of source code management, who will undoubtedly manifest themselves in the coming years.

Roles and code

From a DevOps point of view, it is important to leverage the natural meeting point that a source code management tool is. Many different roles have a use for source code management in its wider meaning. It is easier to do so for technically-minded roles, but harder for other roles, such as project management.

Developers live and breathe source code management. It's their bread and butter.

Operations personnel also favor managing the descriptions of infrastructure in the form of code, scripts, and other artifacts, as we will see in the coming chapters. Such infrastructural descriptors include network topology, and versions of software that should be installed on particular servers.

Quality assurance personnel can store their automated tests in codified form in the source code repository. This is true for software testing frameworks, such as Selenium and JUnit, among others.

There is a problem with the documentation of the manual steps needed to perform various tasks, though. This is more of a psychological or cultural problem than a technical one.

While many organizations employ a wiki solution, such as the wiki engine powering Wikipedia, there is still a lot of documentation floating around in word format on shared drives and in emails.

This makes documentation hard to find and use for some roles and easy for others. From a DevOps viewpoint, this is regrettable, and an effort should be made so that all roles can have good and useful access to the documentation in the organization.

It is possible to store all documentation in the wiki format in the central source code repository, depending on the wiki engine used. Many Git project management systems have this feature now, such as GitLab and GitHub. It is also possible to generate formatted static web pages with, for instance, Hugo, a system for compiling markdown formatted files to websites.

Which source code management system?

There are many **source code management (SCM)** systems out there, and since SCM is such an important part of development, the development of these systems will continue to happen.

Currently, there is a dominant system, however, and that system is Git.

Git has an interesting story: It was initiated by Linus Torvalds to move Linux kernel development from BitKeeper, which was a proprietary system used at the time. The license of BitKeeper changed, so it wasn't practical to use it for the kernel anymore.

Git therefore supports the fairly complicated workflow of Linux kernel development and is, at the base technical level, good enough for most organizations.

The primary benefit of Git versus older systems is that it is a **distributed version control system (DVCS)**. There are many other distributed version control systems, but Git is the most pervasive one.

DVCS has several advantages, including, but not limited to, the following:

- It is possible to use a DVCS efficiently even when not connected to a network. You can take your work with you when traveling on a train or an intercontinental flight.
- Since you don't need to connect to a server for every operation, a DVCS can be faster than the alternatives in most scenarios.
- You can work privately until you feel your work is ready enough to be shared.
- It is possible to work with several remotes simultaneously, which avoids a single point of failure.

Other distributed version control systems apart from Git include the following:

- **Bazaar:** This is abbreviated as bzt. Bazaar is endorsed and supported by Canonical, which is the company behind Ubuntu. Launchpad, which is Canonical's code hosting service, supports Bazaar.
- **Mercurial:** Notable open source projects, such as Firefox and OpenJDK, use Mercurial. It originated around the same time as Git.

Git can be complex, but it makes up for this by being fast and efficient. It can be hard to understand, but that can be made easier by using frontends that support different tasks.

The **version control system (VCS)** used by an organization understandably generates a lot of discussions and sometimes controversy, primarily among developers. Developers use their version control system all day long after all.

A word about source code management system migrations

I have worked with many source code management systems and experienced many transitions from one type of system to another.

Sometimes, much time is spent on keeping all the history intact while performing a migration. For some systems, this effort is well spent, such as for venerable free or open source projects. Emacs, for example, has made several such transitions over the years.

For many organizations, keeping the history is not worth the significant expenditure in time and effort. If an older version is needed at some point, the old source code management system can be kept online and referenced. This includes migrations from Microsoft **Visual SourceSafe** (VSS) and ClearCase.

Some migrations are trivial though, such as moving from Subversion to Git. In these cases, historic accuracy need not be sacrificed.

Choosing a branching strategy

When working with code that deploys to servers, it is important to agree on a branching strategy across the organization.

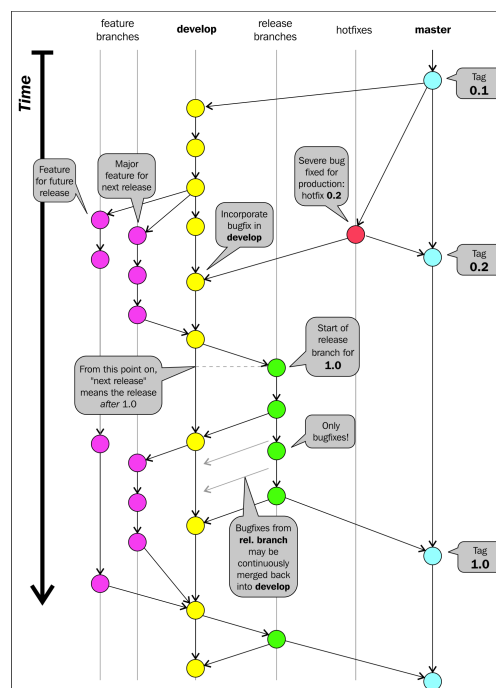
A branching strategy is a convention, or a set of rules, that describes when branches are created, how they are to be named, what use branches should have, and so on.

Branching strategies are important when working together with other people and are, to a degree, less important when you are working on your own, but they still have a purpose.

Most source code management systems do not prescribe a particular branching strategy and neither does Git. The SCM simply gives you the base mechanics to perform branching.

With Git and other distributed VCS, it is usually pretty cheap to work locally with feature branches. A feature, or topic, branch is a branching strategy that is used to keep track of ongoing development regarding a particular feature, bug, and so on. This way, all changes in the code regarding the feature can be handled together.

There are many well-known branching strategies. Vincent Driessen formalized a branching strategy called **Git flow**, which has many good features. For some, Git flow is too complex, and, in those cases, it can be scaled down. There are many such scaled-down models available. This is what Git flow looks like:



Git flow looks complex, so let's have a brief look at what the branches are for:

- The **master** branch only contains finished work. All commits are tagged, since they represent releases. All releases happen from the **master** branch.
- The **develop** branch is where work on the next release happens. When work is finished here, the **develop** branch is merged with the **master** branch.
- We use separate **feature branches** for all new features. **Feature branches** are merged to the **develop** branch.
- When a devastating bug is revealed in production, a **hotfix** branch is made where a bug fix is created. The **hotfix** branch is then merged to the **master** branch, and a new release for production is made.

Git flow is a centralized pattern and, as such, it's reminiscent of the workflows used with Subversion, CVS, and so on. The main difference is that using Git has some technical and efficiency-related advantages.

Another strategy, called the **forking pattern**, where every developer has a central repository, is rarely used in practice within organizations, except when, for instance, external parties such as subcontractors are being employed.

The Top Five Main Git Branching Strategies

Strategy	Overview
GitFlow	Created by Vincent Driessen, GitFlow works with two main branches – master and develop – over the lifetime of the project. It also uses three supporting branches: feature-*, hotfix-*, and release-*. It's the most complex model.
GitHub Flow	A simpler strategy promoted by Scott Chacon of GitHub mandates keeping a continuously deployable main branch. A feature branch is created to work on any feature or bug fix. Each feature branch must be finished and fully tested before being merged with the main branch.
Trunk-Based Development	Very similar to GitHub Flow except that Trunk-Based Development suggests deployment after production code is merged to the main branch to minimize chances for regression.
GitLab Flow	Created by GitLab, this strategy is like an extension of GitHub Flow with master and feature branches. However, it adds environment and release branches to better support SaaS and mobile projects.

How Does Branching Strategy Relate to DevOps?

- DevOps focuses on efficiency – making things easier and faster without compromising quality. Where branching strategy is concerned, DevOps wants to know how pull and merge requests are handled, and the amount of overhead involved in managing the branches.
- “It’s very important to find a branching strategy that fits the way the team works. If branching is not done right, it introduces big burdens and slows the team down. It also can impact the quality of your product. The best guidance is to start small and add complexity only if you need it.”

Branching problem areas

There is a source of contention between **Continuous Delivery (CD)** practices and branching strategies. Some CD methods advocate a single master branch and all releases being made from the master branch. Git flow is one such model.

This simplifies some aspects of deployment, primarily because the branching graph becomes simpler. This, in turn, leads to simplified testing, because there is only one branch that leads to the production servers.

On the other hand, what if we need to perform a bug fix on released code, and the master has new features we don't want to release yet? This happens when the installation cadence in

production is slower than the release cadence of the development teams. It is an undesirable state of affairs, but not uncommon.

There are two basic methods of handling this issue:

- **Make a bug fix branch and deploy to production from it:** This is simpler in the sense that we don't disrupt the development flow. On the other hand, this method might require duplication of testing resources. They might need to mirror the branching strategy.
- **Feature toggling:** Another method that places harder requirements on the developers is feature toggling. In this workflow, you turn off features that are not yet ready for production. This way you can release the latest development version, including the bug fix, together with new features, which will be disabled and inactive.

There are no hard rules, and dogmatism is not helpful in this case.

It is better to prepare for both scenarios and use the method that serves you best in the given circumstances.

Here are some observations to guide you in your choice:

Feature toggling is good when you have changes that are mostly backward compatible or are entirely new. Otherwise, the feature toggling code might become overly complex, catering to many different cases. This, in turn, complicates testing.

Bug fix branches complicate deployment and testing. While it is straightforward to branch and make the bug fix, what should the new version be called, and where do we test it? The testing resources are likely to already be occupied by ongoing development branches.

Some products are simple enough for us to create testing environments on the fly as needed, but this is rarely the case with more complex applications. Sometimes, testing resources are very scarce, such as third-party web services that are difficult to duplicate, or even hardware resources.

MOD-4

Building the Code

You need a system to build your code, and you need somewhere to build it.

This chapter discusses **build systems**, **systems that convert source code into various types of artifacts**, and **systems that check code validity**, among other things. The chapter will cover the following topics:

- Why do we build code? Here, we discuss the purpose of building code.
- The many faces of build systems. Here, we will explore the different build systems and how they affect our DevOps work.
- The Jenkins build server. Jenkins is a flexible open source build server that grows with your needs. Some alternatives to Jenkins will be explored as well.
- Managing build dependencies. You need many bits and pieces to compile code into usable artifacts. These are called build dependencies
- Collating quality measures. Here, we discuss ways of measuring code quality
- Taking build errors seriously. Here, we discuss what to do when code fails to build.

There is a lot to talk about, so let's get started!

Why do we build code?

Most developers are familiar with the process of building code. When we work in the field of DevOps, however, we might face issues that developers who specialize in programming a particular component type won't necessarily experience.

For the purposes of this book, **we define software building as the process of molding code from one form to another.** During this process, several things might happen:

- The **compilation** of source code to native code or virtual machine bytecode, depending on our production platform.
- **Linting** of the code, which involves checking the code for errors and generating code quality measures by means of static code analysis. The term *linting* originated with a program called lint, which started shipping with early versions of the Unix operating system. The purpose of the program was to find bugs in programs that were syntactically correct, but contained suspicious code patterns that could be identified with a different process than compiling.
- **Unit testing**, by running the code in a controlled manner.
- The **generation of artifacts** suitable for deployment.

It's a tall order!

Not all code goes through each and every one of these phases. Interpreted languages, for example, might not need compilation, but they might benefit from quality checks.

The many faces of build systems

There are many build systems that have evolved over the history of software development. Sometimes, it might feel as if there are more build systems than there are programming languages.

Here is a brief list, just to get a feeling for how many there are:

- For Java, there is Maven, Gradle, and Ant
- For C and C++, there is Make in many different flavors

- For Clojure, a language on the **Java Virtual Machine (JVM)**, there is Leiningen and boot, in addition to Maven
- For JavaScript, there is Grunt
- For Scala, there is sbt
- For Ruby, we have Rake
- Finally, of course, we have shell scripts of all kinds

Depending on the size of your organization and the type of product you are building, you might encounter any number of these tools. To make life even more interesting, it's not uncommon for organizations to invent their own build tools.

As a reaction to the complexity of the many build tools, there is also often the idea of standardizing a particular tool. If you are building complex heterogeneous systems, this is rarely efficient. For example, building JavaScript software is just easier with Grunt than it is with Maven or Make, and building C code is not very efficient with Maven. Often, the tool exists for a reason.

Normally, organizations standardize on a single ecosystem, such as Java and Maven, or Ruby and Rake. Other build systems besides those that are used for the primary code base are encountered mainly for native and third-party components.

At any rate, we cannot assume that we will encounter only one build system within our organization's code base, nor can we assume only one programming language.

I have found this rule useful in practice: *it should be possible for a developer to check out the code and build it with minimal surprises on his or her local developer machine.*

This implies that we should standardize the revision control system and have a single interface to start builds locally.

If you have more than one build system to support, this basically means that you need to wrap one build system in another. The complexities of the build are thus hidden and more than one build system at the same time is allowed. Developers not familiar with a particular build can still expect to check it out and build it with reasonable ease.

Maven, for example, is good for declarative Java builds. Maven is also capable of starting other builds from within Maven builds.

This way, the developer in a Java-centric organization can expect the following command line to always build one of the organization's components:

```
mvn clean install
```

One concrete example is creating a Java desktop application installer with the **Nullsoft Scriptable Install System** (NSIS) Windows installation system. The Java components are built with Maven. When the Java artifacts are ready, Maven calls the NSIS installer script to produce a self-contained executable that will install the application on Windows.

While Java desktop applications are not fashionable these days, they continue to be popular in some domains.

The Jenkins build server

A build server is, in essence, a system that builds software based on various triggers. There are several to choose from. In this book, we will have a look at Jenkins, which is a popular build server written in Java.

Jenkins is a fork of the Hudson build server. Kohsuke Kawaguchi was Hudson's principal contributor, and, in 2010, after Oracle acquired Hudson, he continued work on the Jenkins fork. Jenkins is clearly the more successful of the two strains today.

Jenkins has special support for building Java code, but is in no way limited to just building Java. Setting up a basic Jenkins server is not particularly hard at the outset. In Fedora, you can just install it via dnf:

```
dnf install jenkins
```

Jenkins is handled as a service via systemd:

```
systemctl start jenkins
```

You can now have a look at the web interface at <http://localhost:8080>:

The screenshot shows the Jenkins dashboard with a sidebar on the left containing links: New Item, People, Build History, Manage Jenkins, and Credentials. The main area displays a table of jobs. The 'Build Queue' section shows 'No builds in the queue.' and the 'Build Executor Status' section shows two idle executors.

S	W	Name	Last Success	Last Failure	Last Duration
		emacs	50 min - #384	9 mo 8 days - #239	11 min
		fortune	4 days 6 hr - #2	N/A	81 ms
		inkscape	12 hr - #102	10 hr - #103	5 min 16 sec

Icon: [S](#) [M](#) [L](#) [Legend](#) [RSS for all](#) [RSS for failures](#) [RSS for just latest builds](#)

The Jenkins instance in the screenshot has a couple of jobs already defined. The fundamental entity in Jenkins is the job definition, and there are several types to choose from. Let's create a simple job in the web interface. To keep it simple, this job will just print a classic Unix fortune quote:

1. Create a job of the Freestyle project type:

The screenshot shows the 'New Item' form in Jenkins. The 'Item name' field is filled with 'fortunes'. Under the 'Project type' section, 'Freestyle project' is selected. The description for 'Freestyle project' is: 'This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.' Other options include 'Maven project', 'External Job', 'Multi-configuration project', and 'Copy existing Item'. The 'Copy existing Item' option is currently selected, and the 'Copy from' field is empty. An 'OK' button is at the bottom.

2. Add a shell Build step.
3. In the shell entry (Command), type fortune:

[Jenkins](#) > [fortune](#) > [configuration](#)

[Back to Dashboard](#)
[Status](#)
[Changes](#)
[Workspace](#)
[Build Now](#)
[Delete Project](#)
[Configure](#)

Build History

[trend](#)

#2	Sep 6, 2015 1:41 PM
#1	Sep 6, 2015 1:40 PM

[RSS for all](#)
[RSS for failures](#)

Project name
Description

[Escaped HTML] [Preview](#)

☐ Discard Old Builds
☐ This build is parameterized
☐ Disable Build (No new builds will be executed until the project is re-enabled.)
☐ Execute concurrent builds if necessary

Advanced Project Options

Advanced...

Source Code Management

☒ None
☐ Bazaar
☐ CVS
☐ CVS Projectset
☐ Git
☐ Subversion

Build Triggers

☐ Build after other projects are built
☐ Build periodically
☐ Poll SCM

Build

☒ Execute shell

Command

[See the list of available environment variables](#)

Delete

Add build step

Post-build Actions

Add post-build action

Save

Apply

Whenever you run the job, a fortune quote will be printed in the job log.

Jobs can be started manually, and you will find a history of job executions and can examine each job log. This keeps a history of previous executions, which is very handy when you are trying to figure out which change broke a build and how to fix it.

If you don't have the fortune command, install it with `dnf install fortune-mod`, or you might opt to simply run the `date` command instead. This will just output the date in the build log instead of classic quotes and witticisms.

How-to-build-a-Jenkins Build Server

Setup Jenkins Job

Go to Jenkins top page, select “New Job”, then choose “Build a free-style software project”.

Then you can tell the elements of this freestyle job:

- Optional SCM, such as CVS or Subversion where your source code resides.
- Optional triggers to control when Jenkins will perform builds.
- Some sort of build script that performs the build (ant, maven, shell script, batch file, etc.) where the real work happens.
- Optional steps to collect information out of the build, such as archiving the artifacts and/or recording java doc and test results.
- Optional steps to notify other people/systems with the build result, such as sending e-mails, IMs, updating issue tracker, etc.

Managing build dependencies

In the previous, simple example, we printed a fortune cookie to the build log.

While this exercise can't be compared in complexity with managing real builds, we at least learned to install and start Jenkins, and if you had issues installing the fortune utility, you got a glimpse of the dark underbelly of managing a CI server: managing the build dependencies.

Some build systems, such as the Maven tool, are nice in the way that the Maven POM file contains descriptions of which build dependencies are needed, and they are fetched automatically by Maven if they aren't already present on the build server. Grunt works in a similar way for JavaScript builds. There is a build description file that contains the dependencies required for the build. Golang builds can even contain links to GitHub repositories required for completing the build.

C and C++ builds present challenges in a different way. Many projects use GNU Autotools; among them is Autoconf, which adapts itself to the dependencies that are available on the host rather than describing which dependencies they need. So, to build Emacs, a text editor, you first run a configuration script that determines which of the many potential dependencies are available on the build system.

If an optional dependency is missing, such as image libraries for image support, the optional feature will not be available in the final executable. You can still build the program, but you won't get features that your build machine isn't prepared for.

While this is a useful feature, if you want your software to work in many different configurations depending on which system it should run on, it's not often the way we would like our builds to behave in an enterprise setting. In this case, we need to be perfectly sure which features will be available in the end. We certainly don't want bad surprises in the form of missing functionality on our production servers.

The **Red Hat Package Manager (RPM)** system, which is used on systems derived from Red Hat, offers a solution to this problem (RPM Package Manager is a free and open-source package management system. The name RPM refers to .rpm file format and the package manager program itself. RPM was intended primarily for Linux distributions; the file format is the baseline package format of the Linux Standard Base. An RPM package typically contains binary

executables, along with relevant configuration files and documentation. The rpm program is a powerful package manager, which can be used to install, query, verify, update, erase and build software packages in the RPM format.). At the core of the RPM system is a build descriptor file called a spec file, short for specification file. It lists, among other things, the build dependencies required for a successful build and the build commands and configuration options used. Since a spec file is essentially a macro-based shell script, you can use it to build many types of software. The RPM system also has the idea that build sources should be pristine. The spec file can adapt the source code by patching the source before building it.

A Project Object Model or POM is the fundamental unit of work in Maven. It is an XML file that contains information about the project and configuration details used by Maven to build the project.

- It contains default values for most projects.
- Examples for this are the build directory, which is target; the source directory, which is src/main/java; the test source directory, which is src/test/java; and so on. When executing a task or goal, Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, then executes the goal.
- Some of the configurations that can be specified in the POM are the project dependencies, the plugins or goals that can be executed, the build profiles, and so on.
- Other information such as the project version, description, developers, mailing lists and such can also be specified.

The final artifact

After finishing the build using the RPM system, you get an RPM file, which is a very convenient type of deployment artifact for operating systems based on Red Hat.

For Debian-based distributions, you get a .deb file.

The final output from a Maven build is usually an enterprise archive, or EAR file for short. This contains Java Enterprise applications.

It is final deployment artifacts such as these that we will later deploy to our production servers.

In this chapter, we concern ourselves with building the artifacts required for deployment, and, in [Chapter 7, *Deploying the Code*](#), we talk about the final deployment of our artifacts.

However, even when building our artifacts, we need to understand how to deploy them. At the moment, we will use the following rule of thumb: operating system-level packaging is preferable to specialized packaging. This is my personal preference, and others might disagree.

Let's briefly discuss the background for this rule of thumb, as well as the alternatives.

As a concrete example, let's consider the deployment of a Java **Enterprise Application Archive (EAR)**. Normally, we can do this in several ways. Here are some examples:

- Deploy the EAR file as an RPM package through the mechanisms and channels available in the base operating system
- Deploy the EAR file through the mechanisms available with the Java application server, such as JBoss, WildFly, and GlassFish

It might superficially look like it would be better to use the mechanism specific to the Java application server to deploy the EAR file, since it is specific to the application server anyway. If Java development is all you ever do, this might be a reasonable supposition. However, since you need to manage your base operating system anyway, you already have methods of deployment available to you that are possible to reuse.

Also, since it is quite likely that you are not just doing Java development but also need to deploy and manage HTML and JavaScript at the very least, it starts to make sense to use a more versatile method of deployment.

Nearly all the organizations I have experience of have had complicated architectures comprising many different technologies, and this rule of thumb has served well in most scenarios.

The only real exception is in mixed environments where Unix servers coexist with Windows servers. In these cases, the Unix servers usually get to use their preferred package distribution

method, and the Windows servers have to limp along with some kind of home-brewed solution. This is just an observation and not a condoning of the situation.

Another method altogether is to use a container-based system such as Docker to deploy your application. This is discussed elsewhere in this book.

Continuous Integration

The principal benefit of using a build server is achieving **Continuous Integration (CI)**. Each time a change in the code base is detected, a build that tests the quality of the newly submitted code is started.

Since there might be many developers working on the code base, each with slightly different versions, it's important to see whether all the different changes work together properly. This is called **integration testing**. If integration tests are too far apart, there is a growing risk of the different code branches diverging too much, and merging is no longer easy. The result is often referred to as **merge hell**. It's no longer clear how a developer's local changes should be merged to the master branch, because of divergence between the branches. This situation is very undesirable. The root cause of merge hell is often, perhaps surprisingly, psychological. There is a mental barrier to overcome in order to merge your changes to the mainline. Part of working with DevOps is making things easier and thus reducing the perceived costs associated with doing important work, such as submitting changes.

CI builds are usually performed in a more stringent manner than what developers do locally. These builds take a longer time to perform, but since performance hardware is not so expensive these days, our build server is beefy enough to cope with these builds.

If the builds are fast enough to not be seen as tedious, developers will be enthused to check in often, and integration problems will be found early.

Continuous Delivery

After the CI steps have completed successfully, you will have shiny new artifacts that are ready to be deployed to servers. Usually, these are test environments set up to behave like production servers. We will discuss deployment system alternatives later in the book.

Often, the last thing a build server does is to deploy the final artifacts from the successful build to an artifact repository. From there, the deployment servers take over the responsibility of deploying them to the application servers. In the Java world, the Nexus repository manager is fairly common. It has support for other formats besides the Java formats, such as JavaScript artifacts and Yum channels for RPMs. Nexus also supports the Docker Registry API now.

Using Nexus for RPM distributions is just one option. You can build Yum channels with a shell script fairly easily.

Artifact repository is a collection of binary software artifacts and metadata stored in a defined directory structure which is used by clients such as Maven, Mercury, or Ivy to retrieve binaries during a build process. A Maven repository provides a platform for the storage, retrieval, and management of binary software artifacts and metadata.

The host server

The build server is usually a pretty important machine for the organization. Building software is processor-intensive, as well as memory- and disk-intensive. Builds shouldn't take too long, so you will need a server with good specifications for the build server with lots of disk space, processor cores, and RAM.

The build server also has a kind of social aspect: it is here that the code of many different people and roles integrates properly for the first time. This aspect grows in importance if the servers are fast enough. Machines are cheaper than people, so don't let this particular machine be the area you save money on.

Build slaves

To reduce build queues, you can add build slaves. The master server will send builds to the slaves based on a round-robin scheme or tie specific builds to specific build slaves.

The reason for this is usually that some builds have certain requirements on the host OS.

Build slaves can be used to increase the efficiency of parallel builds. They can also be used to build software on different operating systems. For instance, you can have a Linux Jenkins master server and Windows slaves for components that use Windows build tools. To build software for the Apple Mac, it's useful to have a Mac build slave, especially since Apple has quirky rules regarding the deployment of their OS on virtual servers.

There are several methods to add build slaves to a Jenkins master; see

<https://wiki.jenkins-ci.org/display/JENKINS/Distributed+builds>.

In essence, there must be a way for the Jenkins master to issue commands to the build slave. This command channel can be the classic SSH method, and Jenkins has a built-in SSH facility. You can also start a Jenkins slave by downloading a Java **Java Network Launch Protocol (JNLP)** client from the master to the slave. This is useful if the build slave doesn't have an SSH server.

A note on cross-compiling:

While it's possible to use Windows build slaves, sometimes it's actually easier to use Linux to build Windows software. C compilers, such as GCC, can be configured to perform cross-compilation using the MinGW package. Whether or not this is easier very much depends on the software being built.

A big system usually comprises many different parts, and some of the parts might contain native code for different platforms.

Here are some examples:

Native Android components

Native server components coded in C for efficiency

Native client components, also coded in C or C++ for efficiency

The prevalence of native code depends a bit on the nature of the organization you are working with. Telecom products often have a lot of native code, such as codecs and hardware interface code. Banking systems might have high-speed messaging systems in native code.

One aspect of this is that it is important to be able to build all the code that's in use conveniently on the build server. Otherwise, there's a tendency for some code to be only buildable on some machine collecting dust under somebody's desk. This is a risk that needs to be avoided.

What your organization's systems need, only you can tell.

Software on the host

Depending on the complexity of your builds, you might need to install many different types of build tool on your build server. Remember that Jenkins is mostly used to trigger builds, not perform the builds themselves. That job is delegated to the build system used, such as Maven or Make.

In my experience, it's most convenient to have a Linux-based host OS. Most of the build systems are available in the distribution repositories, so it's very convenient to install them from there.

To keep your build server up to date, you can use the same deployment servers that you use to keep your application servers up to date.

Jenkins also has a method to deploy some builders, such as Maven, on the build slave. This is convenient when the builder you are using exists as a Jenkins plugin, but that's not always the case.

Triggers

You can either use a timer to trigger builds, or you can poll the code repository for changes and build if there were changes.

It can be useful to use both methods at the same time:

- Git repository polling can be used most of the time so that every check in triggers a build.
- Nightly builds can be triggered, which are more stringent than continuous builds, and thus take a longer time. Since these builds happen at night when nobody is supposed to work, it doesn't matter if they are slow.
- An upstream build can trigger a downstream build.

You can also let the successful build of one job trigger another job.

Job chaining and build pipelines

It's often useful to be able to chain jobs together. In its simplest form, this works by triggering a second job in the event that the first job finishes successfully. Several jobs can be cascaded this way in a chain. Such a build chain is quite often good enough for many purposes. Sometimes, a nicer visualization of the build steps, as well as greater control over the details of the chain, is desired.

In Jenkins terminology, the first build in a chain is called the upstream build, and the second one is called the downstream build.

While this way of chaining builds is often sufficient, there will most likely be a need for greater control of the build chain eventually. Such a build chain is often called a pipeline or workflow.

There are many plugins that create improved pipelines for Jenkins, and the fact that there are several shows that there is indeed a great desire for improvements in this area.

Examples include the multi-job plugin, the workflow plugin, and the pipeline plugin.

The pipeline plugin is the more advanced and also has the advantage that it can be described in a Groovy DSL rather than fiddling about in a web UI. It also supports storing the textual pipeline description in a file called a Jenkinsfile, which can be version control-managed like other sources. It is promoted by CloudBees, who are the principal contributors to Jenkins today.

The pipeline plugin comes with a couple of hello world style examples. Here is a variant that checks out code using Git and then builds it using Maven:

```
node {
    def mvnHome
    stage('Preparation') { // for display purposes
        // Get some code from a GitHub repository
                                                                    git
        'https://github.com/jglick/simple-maven-project-with-tests.git'
        // Get the Maven tool.
        // ** NOTE: This 'M3' Maven tool must be configured
        // **      in the global configuration.
        mvnHome = tool 'M3'
    }
    stage('Build') {
        // Run the maven build
        if (isUnix()) {
            sh "'${mvnHome}/bin/mvn' -Dmaven.test.failure.ignore
clean package"
        } else {
            bat ("\"${mvnHome}\\bin\\mvn\" -Dmaven.test.failure.ignore
clean package/")
        }
    }
}
```

```

stage('Results') {
    junit '**/target/surefire-reports/TEST-*.xml'
    archive 'target/*.jar'
}
}

```

When you have a look at the build script that the pipeline plugin uses, you might get the idea that Jenkins is basically a build tool with a web interface, and you would be more or less correct.

When the pipeline runs, you get a view like the following:

Jenkins

Jenkins > mypipeline

[Back to Dashboard](#)
[Status](#)
[Changes](#)
[Build Now](#)
[Delete Pipeline](#)
[Configure](#)
[Full Stage View](#)
[Pipeline Syntax](#)

Pipeline mypipeline

[Last Successful Artifacts](#)
 simple-maven-project-with-tests-1.0-SNAPSHOT.jar 1,71 KB [view](#)
[Recent Changes](#)

Stage View

	Preparation	Build	Results
Average stage times:	8s	4s	467ms
#2 Jan 28 23:09 No Changes	8s	4s	467ms

[RSS for all](#) [RSS for failures](#)

[Latest Test Result](#) (1 failure)

Permalinks

- [Last build \(#2\), 6 min 17 sec ago](#)
- [Last successful build \(#2\), 6 min 17 sec ago](#)
- [Last unstable build \(#2\), 6 min 17 sec ago](#)
- [Last unsuccessful build \(#2\), 6 min 17 sec ago](#)
- [Last completed build \(#2\), 6 min 17 sec ago](#)

Build servers and Infrastructure as Code

While we are discussing the Jenkins file structure, it is useful to note an impedance mismatch that often occurs between GUI-based tools, such as Jenkins, and the DevOps axiom that infrastructure should be described as code.

One way to understand this problem is that while Jenkins job descriptors are text file-based, these text files are not the primary interface for changing the job descriptors. The web interface is the primary interface. This is both a strength and a weakness.

It is easy to create ad-hoc solutions on top of existing builds with Jenkins. You don't need to be intimately familiar with Jenkins to do useful work.

On the other hand, the out-of-the-box experience of Jenkins lacks many features that we are used to from the world of programming. Basic features such as inheritance and even function definitions take some effort to provide in Jenkins.

The build server feature in GitLab, for example, takes a different approach. Build descriptors are just code right from the start. It is worth checking out this feature in GitLab if you don't need all the possibilities that Jenkins offers.

Build phases

One of the principal benefits of the Maven build tool is that it standardizes builds.

This is very useful for a large organization, since it won't need to invent its own build standards.

Other build tools are usually much more lax regarding how to implement various build phases.

The rigidity of Maven has its pros and cons. Sometimes, people who get started with Maven reminisce about the freedom that could be had with tools such as Ant.

You can implement these build phases with any tool, but it's harder to keep the habit going when the tool itself doesn't enforce the standard order: building, testing, and deploying.

We will examine testing in more detail in a later chapter, but we should note here that the testing phase is very important. The CI server needs to be very good at catching errors, and automated testing is very important for achieving that goal.

Collating quality measures

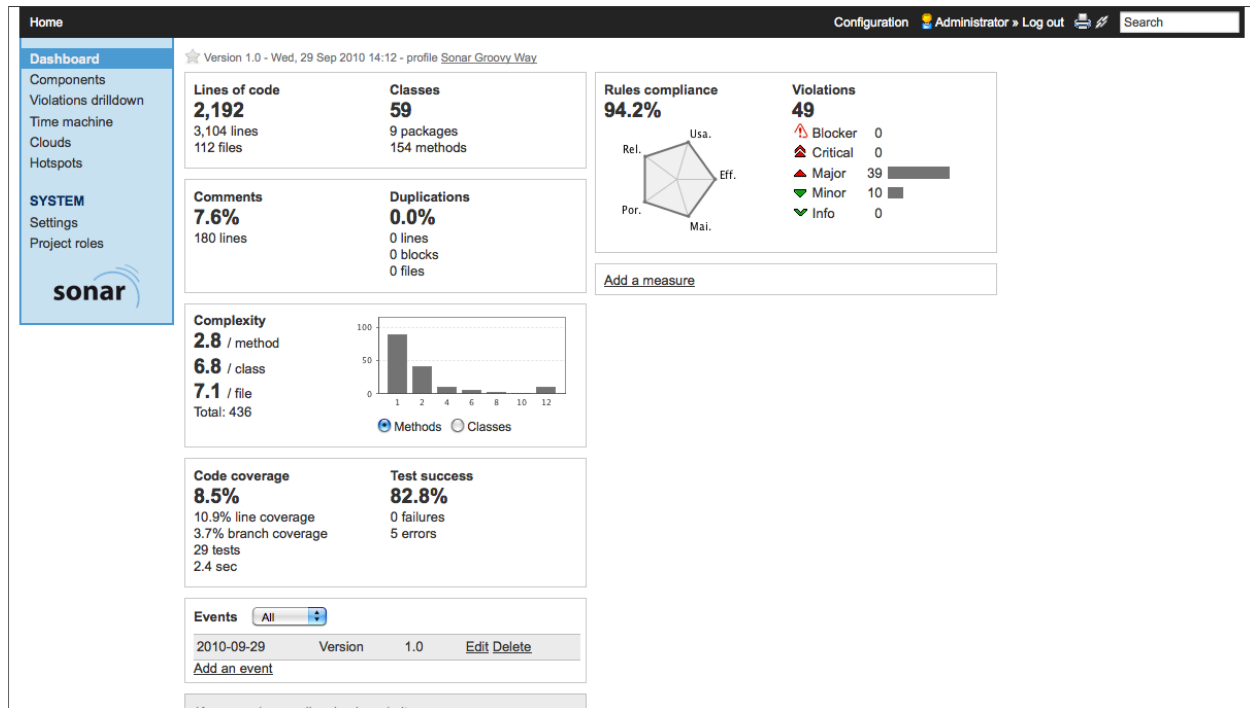
A useful thing that a build server can do is the collation of software quality metrics. Jenkins has some support for this out of the box. Java unit tests are executed and can be visualized directly on the job page.

Another more advanced option is using the Sonar code quality visualizer, which is shown in the following screenshot. Sonar tests are run during the build phase and propagated to the Sonar server, where they are stored and visualized.

A Sonar server can be a great way for a development team to see the fruits of their efforts at improving the code base.

The drawback of implementing a Sonar server is that it sometimes slows down the builds. The recommendation is to perform the Sonar builds in your nightly builds, once a day.

The following screenshot shows some of Sonar's many different code metrics, from simple ones like the number of lines of code, through various complexity measures, to rule compliance:



About build status visualization

The build server produces a lot of data that is amenable to visualization on a shared display. It is useful to be immediately aware that a build has failed, for instance.

The easiest thing is to just hook up a monitor in a kiosk-like configuration with a web browser pointing to your build server web interface. Jenkins has many plugins that provide a simplified job overview suitable for kiosk displays. These are sometimes called **information radiators**.

It is also common to hook up other types of hardware to the build status, such as lava lamps or colorful LED lamps.

In my experience, this kind of display can make people enthusiastic about the build server. Succeeding with having a useful display in the long run is more tricky than it would first appear,

though. The screen can be distracting. If you put the screen where it's not easily seen in order to circumvent the distraction, the purpose of the display is defeated.

A lava lamp in combination with a screen placed discreetly could be a useful combination. The lava lamp is not normally lit, and thus not distracting. When a build error occurs, it lights up, and then you know that you should have a look at the build information radiator. The lava lamp even conveys a form of historical record of the build quality. As the lava lamp lights up, it grows warm, and after a while, the lava moves around inside the lamp. When the error is corrected, the lamp cools down, but the heat remains for a while, so the lava will move around for a time proportional to how long it took to fix the build error.

Robustness

While it is desirable that the build server becomes one of the focal points in your CD pipeline, also consider that the process of building and deployment should not come to a standstill in the event of a breakdown of the build server. For this reason, the builds themselves should be as robust as possible and repeatable on any host.

This is fairly easy for some builds, such as Maven builds. Even so, a Maven build can exhibit any number of flaws that makes it non-portable.

A C-based build can be pretty hard to make portable if one is not so fortunate as to have all build dependencies available in the operating system repositories. Still, robustness is usually worth the effort.

Testing the Code

If we are going to release our code early and often, we ought to be confident of its quality. Therefore, we need automated regression testing.

In this chapter, some frameworks for software testing are explored, such as **JUnit** for unit testing and **Selenium** for web frontend testing. We will also find out how these tests are run in our **Continuous integration (CI)** server, Jenkins, thus forming the first part of our **Continuous delivery (CD)** pipeline. Testing is very important for software quality, and it's a very large subject in itself.

We will concern ourselves with the following topics in this chapter:

- How to make manual testing easier and less error prone
- Various types of testing, such as unit testing, and how to perform them in practice
- Automated system integration testing

We've already had a look at how to accumulate test data with Sonar and Jenkins in the previous chapter, [Chapter 5](#), *Building the Code*, and we will continue to delve deeper into this subject.

Manual testing

Even if test automation has larger potential benefits for DevOps than manual testing, manual testing will always be an important part of software development. If nothing else, we will need to perform our tests manually at least once in order to automate them.

Acceptance testing in particular is hard to replace, even though there have been attempts to do so. Software requirement specifications can be terse and hard to understand, even for the people developing the features that implement those requirements. In these cases, quality assurance people with their eyes on the ball are invaluable and irreplaceable.

The things that make manual testing easier are the same things that make automated integration testing easier as well, so there is a synergy to achieve between the different testing strategies.

In order to have happy quality assurance personnel, you need to:

- Manage test data, primarily the contents of backend databases, so that tests give the same results when you run them repeatedly
- Be able to make rapid deployments of new code in order to verify bug fixes

Obvious as this may seem, it can be hard in practice. Maybe you have large production databases that can't just be copied to test environments. Maybe they contain end-user data that needs to be protected under law. In these cases, you need to de-identify the data and wash it of any personal details before deploying it to test environments.

Each organization is different, so it is not possible to give generally useful advice in this area other than the KISS rule: *keep it simple, stupid*.

Pros and cons with test automation

When you talk with people, most are enthusiastic about the prospect of test automation. Imagine all the benefits that await us with it:

- Higher software quality
- Higher confidence that the software releases we make will work as intended
- Less of the monotonous tedium of laborious manual testing

All very good and desirable things!

In practice, though, if you spend time with different organizations with complex multi-tiered products, you will notice people talking about test automation, but you will also notice a suspicious absence of test automation in practice. Why is that?

If you just compile programs and deploy them once they pass compilation, you will likely be in for a bad experience. Software testing is completely necessary for a program to work reliably in

the real world. Manual testing is too slow to achieve CD. So, we need test automation to succeed with CD. Therefore, let's further investigate the problem areas surrounding test automation and see if we can figure out what to do to improve the situation:

- Cheap tests have lower value:
 - One problem is that the type of test automation that is fairly cheap to produce, unit testing, typically has lower perceived value than other types of testing. Unit testing is still a good type of testing, but manual testing might be perceived as exposing more bugs in practice. It might then feel unnecessary to write unit tests.
- It is difficult to create test cradles that are relevant to automated integration testing:
 - While it is not very difficult to write test cradles or test fixtures for unit tests, it tends to get harder as the test cradle becomes more production-like. This can be because of a lack of hardware resources, licensing, manpower, and so on.
- The functionality of programs vary over time and tests must be adjusted accordingly, which takes time and effort:
 - This makes it seem as though test automation just makes it harder to write software without providing a perceived benefit.
 - This is especially true in organizations where developers don't have a close relationship with the people working with operations, that is, a non DevOps-oriented organization. If someone else will have to deal with your crappy code that doesn't really work as intended, there is no real cost involved for the developers. This isn't a healthy relationship. This is the central problem DevOps aims to solve. The DevOps approach bears repeating: help people with different roles work closer together. In organizations such as Netflix, an Agile team is entirely responsible for the success, maintenance, and outages of their service.

- It is difficult to write robust tests that work reliably in many different build scenarios:
 - A consequence of this is that developers tend to disable tests in their local builds so that they can work undisturbed with the feature they have been assigned. Since people don't work with the tests, changes that affect the test outcomes creep in, and eventually, the tests fail.
 - The build server will pick up the build error, but nobody remembers how the test works now, and it might take several days to fix the test error. While the test is broken, the build displays will show red, and eventually, people will stop caring about build issues. Someone else will fix the problem eventually.
- It is just hard to write good automated tests, period.
- It can indeed be hard to create good automated integration tests. It can also be rewarding, because you get to learn all the aspects of the system you are testing.
- These are all difficult problems, especially since they mostly stem from people's perceptions and relationships.

There is no panacea, but I suggest adopting the following strategy:

- Leverage people's enthusiasm regarding test automation
- Don't set unrealistic goals
- Work incrementally

Unit testing

Unit testing is the sort of testing that is normally close at heart for developers. The primary reason is that, by definition, unit testing tests well-defined parts of the system in isolation from other parts. Thus, they are comparatively easy to write and use.

Many build systems have built-in support for unit tests, which can be leveraged without undue difficulty.

With Maven, for example, there is a convention that describes how to write tests such that the build system can find them, execute them, and finally prepare a report of the outcome. Writing tests basically boils down to writing test methods, which are tagged with source code annotations to mark the methods as being tests. Since they are ordinary methods, they can do anything, but by convention, the tests should be written so that they don't require considerable effort to run. If the test code starts to require complicated setup and runtime dependencies, we are no longer dealing with unit tests.

Here, the difference between unit testing and functional testing can be a source of confusion. Often, the same underlying technologies and libraries are reused between unit and functional testing.

This is a good thing, as reuse is good in general and lets you benefit from your expertise in one area as you work on another. Still, it can be confusing at times, and it pays to raise your eyes now and then to see that you are doing the right thing.

xUnit in general and JUnit in particular

You need something that runs your tests. JUnit is a framework that lets you define unit tests in your Java code and run them.

JUnit belongs to a family of testing frameworks collectively called **xUnit**. This is the grandfather of this family and was designed by Kent Beck in 1998 for the Smalltalk language.

While JUnit is specific to Java, the ideas are sufficiently generic for ports to have been made in, for instance, C#. The corresponding test framework for C# is called, somewhat unimaginatively, **NUnit**. The N is derived from .NET, the name of the Microsoft software platform.

We need some of the following nomenclature before carrying on. The nomenclature is not specific to JUnit, but we will use JUnit as an example to make it easier to relate to the definitions:

- **Test runner:** A test runner runs tests that are defined by an xUnit framework:
 - JUnit has a way to run unit tests from the command line, and Maven employs a test runner called Surefire. A test runner also collects and reports test results. In the case of Surefire, the reports are in XML format, and these reports can be further processed by other tools, particularly for visualization.
- **Test case:** A test case is the most fundamental type of test definition:
 - How you create test cases differs a little bit among JUnit versions. In earlier versions, you inherited from a JUnit base class; in recent versions, you just need to annotate the test methods. This is better since Java doesn't support multiple inheritance and you might want to use your own inheritance hierarchies rather than the JUnit ones. By convention, Surefire also locates test classes that have the Test suffix in the class name.
- **Test fixtures:** A test fixture is a known state that the test cases can rely on so that the tests can have well-defined behavior. It is the responsibility of the developer to create these. A test fixture is also sometimes known as a test context:
 - With JUnit, you usually use the `@Before` and `@After` annotations to define test fixtures. `@Before` is, unsurprisingly, run before a test case and is used to bring up the environment. `@After`, likewise, restores the state if there is a need to.
 - Sometimes, `@Before` and `@After` are more descriptively named **setup** and **teardown**. Since annotations are used, the method can have the names that are the most intuitive in that context.

- **Test suites:** You can group test cases together in test suites. A test suite is usually a set of test cases that share the same test fixture.
- **Test execution:** A test execution runs the test suites and test cases:
 - Here, all the previous aspects are combined. The test suites and test cases are located, the appropriate test fixtures are created, and the test cases run. Lastly, the test results are collected and collated.
- **Test result formatter:** A test result formatter formats test result output for human consumption. The format employed by JUnit is versatile enough to be used by other testing frameworks and formatters not directly associated with JUnit. So, if you have some tests that don't really use any of the xUnit frameworks, you can still benefit by presenting the test results in Jenkins by providing a test result XML file. Since the file format is XML, you can produce it from your own tool, if need be.
- **Assertions:** An assertion is a construct in the xUnit framework that makes sure that a condition is met. If it is not met, it is considered an error, and a test error is reported. The test case is also usually terminated when the assertion fails.

JUnit has a number of assertion methods available. Here is a sample of the available assertion methods:

- To check whether two objects are equal:

```
assertEquals(str1, str2);
```

- To check whether a condition is True:

```
assertTrue (val1 < val2);
```

- To check whether a condition is False:

```
assertFalse(val1 > val2);
```

Mocking

Mocking refers to the practice of writing simulated resources to enable unit testing. Sometimes, the words **fake** or **stub** are used. For example, a middleware system that responds with JSON structures from a database would mock the database backend for its unit tests. Otherwise, the unit tests would require the database backend to be online, probably also requiring exclusive access. This wouldn't be convenient.

Mockito is a mocking framework for Java that has also been ported to Python.

Test coverage

When you hear people talk about unit testing, they often talk about test coverage. **Test coverage is the percentage of the application code base that is actually executed by the test cases.**

In order to measure unit test code coverage, you need to execute the tests and keep track of the code that has or hasn't been executed.

Cobertura is a test coverage measurement utility for Java that does this. Other such utilities include jcoverage and Clover.

Cobertura works by instrumenting the Java bytecode, inserting code fragments of its own into already compiled code. These code fragments are executed while measuring code coverage during execution of test cases

It's usually assumed that a hundred percent test coverage is the ideal. This might not always be the case, and you should be aware of the cost/benefit trade-offs.

A simple counter-example is a simple getter method in Java:

```
private int positiveValue;

void setPositiveValue(int x){
    this.positiveValue=x;
}

int getPositiveValue(){
```

```
return positiveValue;
```

```
}
```

If we write a test case for this method, we will achieve a higher test coverage. On the other hand, we haven't achieved much of anything, in practice. The only thing we are really testing is that our Java implementation doesn't have bugs.

If, on the other hand, the setter is changed to include validation to check that the value is not a negative number, the situation changes. As soon as a method includes logic of some kind, unit testing is useful.

A complete test automation scenario

We have looked at a number of different ways of working with test automation. Assembling the pieces into a cohesive whole can be daunting.

In this section, we will have a look at a complete test automation example, continuing from the user database web application for our organization, Matangle.

You can find the source code in the accompanying source code bundle for the book.

The application consists of the following layers:

- A web frontend
- A JSON/REST service interface
- An application backend layer
- A database layer

The test code will work through the following phases during execution:

1. Unit testing of the backend code.

2. Functional testing of the web frontend, performed with the Selenium web testing framework.
3. Functional testing of the JSON/REST interface, executed with SoapUI.

All the tests are run in sequence, and when all of them succeed, the result can be used as the basis for a decision to see whether the application stack is deemed healthy enough to deploy to a test environment, where manual testing can commence.

MOD-5

Deploying the Code

Now that the code has been built and tested, we need to deploy it to our servers so that our customers can use these newly developed features!

There are many competing tools and options in this space, and the one that is right for you and your organization will depend on your needs.

We will explore Puppet, Ansible, Salt, and others, by showing you how to deploy sample applications in different scenarios. Any one of these tools has a vast ecosystem of complementary services and tools, so it is not an easy subject to get a grip on.

Throughout the book, we have encountered aspects of some of the different deployment systems that already exist. We had a look at RPMs and .deb files and how to build them with the `rpm` command. We had a look at various Java artifacts and how Maven uses the idea of a binary repository where you can deploy your versioned artifacts.

In this chapter, we will focus on installing binary packages and their configuration with a configuration management system. We will also cover the following topics:

- Why are there so many deployment systems?
- Virtualization stacks
- Puppet, Ansible, Salt, and Chef
- Docker
- Cloud solutions

Why are there so many deployment systems?

There is a bewildering abundance of options regarding the installation of packages and configuring them on actual servers, not to mention all the ways to deploy client-side code.

First, let's examine the basics of the problem we are trying to solve.

We have a typical enterprise application, with a number of different high-level components. We don't need to make the scenario overly complex to start reasoning about the challenges that exist in this space.

In our scenario, we have the following:

- A web server
- An application server
- A database server

If we only have a single physical server and these few components to worry about that get released once a year or so, we can install the software manually and be done with the task. It will be the most cost-effective way of dealing with the situation, even though manual work is boring and error-prone.

It's not reasonable to expect a conformity to this simplified release cycle in reality, though. It is more likely that a large organization has hundreds of servers and applications and that they are all deployed differently, with different requirements.

Managing all of the complexity that the real world displays is hard, so it starts to make sense that there are a lot of different solutions that do basically the same thing in different ways.

Whatever the fundamental unit that executes our code is, be it a physical server, a **virtual machine (VM)**, some form of container technology, or a combination of these, we have several challenges to deal with. We will look at them now.

Configuring the base OS

The configuration of the base **operating system (OS)** must be dealt with somehow.

Often, our application stack has subtle, or not so subtle, requirements on the base OS. Some application stacks, such as Java, Python, or Ruby, make these OS requirements less apparent, because these technologies go to great lengths to offer cross-platform functionality. At other times, the OS requirements are apparent to a greater degree, such as when you work with low-level mixed hardware and software integrations, which is common in the telecom industry.

There are many existing solutions that deal with this fundamental issue. Some systems work with a bare metal (or bare VM) approach, where they install the desired OS from scratch and then install all the base dependencies that the organization needs for their servers. Such systems include, Red Hat Satellite and Cobbler, which work in a similar way but are more lightweight.

Cobbler allows you to boot a physical or VM over the network using `dhcpd`. The **Dynamic Host Configuration Protocol (DHCP)** server can then allow you to provide a netboot-compliant image. When the netboot image is started, it contacts Cobbler to retrieve the packages that will be installed to create the new OS. Which packages are installed can be decided on the server from the target machine's network MAC address, for instance.

Another method that is very popular today is to provide base OS images that can be reused between machines. Cloud systems such as **Amazon Web Services (AWS)**, Azure, or OpenStack work this way. When you ask the cloud system for a new VM, it is created using an existing image as a base. Container systems such as Docker also work in a similar way, where you declare your base container image and then describe the changes you want to formulate for your own image.

Describing clusters

There must be a way to describe clusters.

If your organization only has a single machine with a single application, then you may not need to describe what a cluster deployment of your application would look like. Unfortunately (or

fortunately, depending on your outlook), the reality is normally that your applications are spread out over a set of machines, virtual or physical.

All of the systems that we'll be working with in this chapter support this idea in different ways. Puppet has an extensive system that allows machines to have different roles that, in turn, imply a set of packages and configurations. Ansible and Salt have these systems as well. The container-based Docker system has an emerging infrastructure for describing sets of containers connected together and Docker hosts that can accept and deploy such cluster descriptors.

Cloud systems such as AWS also have methods and descriptors for cluster deployments. Cluster descriptors are normally also used to describe the application layer.

Delivering packages to a system

There must be a way to deliver packages to a system.

Much of an application can be installed as packages, which are installed—unmodified—on the target system by the configuration management system. Package systems such as RPM and deb have useful features, such as verifying that the files provided by a package have not been tampered with on a target system, by providing checksums for all files in the package. This is useful for security reasons as well as for debugging purposes. Package delivery is usually done with OS facilities such as yum package channels on Red Hat-based systems, but, sometimes, the configuration management system can also deliver packages and files with its own facilities. These facilities are often used in tandem with the OS's package channels.

There must be a way to manage configurations that are independent of installed packages.

The configuration management system should, obviously, be able to manage our applications' configurations. This is complex because configuration methods vary wildly between applications, regardless of the many efforts that have been made to unify the field.

The most common and flexible system to configure applications relies on text-based configuration files. There are several other methods, such as using an application that provides an API to handle configuration (such as a command-line interface), or sometimes handling configuration via database settings.

In my experience, configuration systems based on text files create the least amount of hassle and should be preferred for in-house code at least. There are many ways to manage text-based configurations. You can manage them in source code-handling systems such as Git. There's a host of tools that can ease the debugging of broken configuration, such as diff. If you are in a tight spot, you can edit configurations directly on the servers using a remote text editor such as Emacs or vi.

Handling configurations via databases is much less flexible. This is arguably an anti-pattern that usually occurs in organizations where the psychological rift between developer teams and operations teams is too wide, which is something we aim to solve with DevOps. Handling configurations in databases makes the application stack harder to get running. You need a working database to even start the application.

Managing configuration settings via imperative command-line APIs is also a dubious practice for similar reasons but can sometimes be helpful, especially if the API is used to manage an underlying text-based configuration. Many of the configuration management systems, such as Puppet, depend on being able to manage declarative configurations. If we manage the configuration state via other mechanisms, such as command-line imperative API, Puppet loses many of its benefits.

Even managing text-based configuration files can be a hassle. There are many ways for applications to invent their own configuration file formats, but there is a set of base file formats that are popular. Such file formats include XML, YML, JSON, and INI/TOML.

Usually, configuration files are not static, because if they were, you could just deploy them with your package system like any piece of binary artifact.

Normally, the application configuration files need to be based on some kind of template file that is later instantiated into a form suitable for the machine where the application is being deployed.

An example could be an application's database connector descriptor. If you are deploying your application to a test environment, you want the connector descriptor to point to a test database server. Vice versa, if you are deploying to a production server, you want your connector to point to a production database server.

As an aside, some organizations try to handle this situation by managing their DNS servers, so that an example database DNS alias `database.yourorg.com` resolves to different servers depending on the environment. The `yourorg.com` domain should be replaced with your organization's details, of course, and the database server as well.

Being able to use different DNS resolvers depending on the environment is a useful strategy. It can be difficult for a developer, however, to use the equivalent mechanism on their own development machine. Running a private DNS server on a development machine can be difficult, and managing a local host file can also prove cumbersome. In these cases, it may be simpler to make the application have configurable settings for database hosts and other backend systems at the application level.

In most cases, it is possible to ignore the details of the actual configuration file format altogether and just rely on the configuration system's template managing system. These usually work by having a special syntax for placeholders that will be replaced by the configuration management system when creating the concrete configuration file for a concrete server, where the application will be deployed. You can use the exact same basic idea for all text-based configuration files, and sometimes even for binary files, even though such hacks should be avoided if at all possible.

The XML format has tools and infrastructure that can be useful in managing configurations, and XML is indeed a popular format for configuration files. For instance, there is a special language, XSLT, to transform XML from one structural form to another. This is very helpful in some cases,

but used less in practice than you may expect. The simple template macro substitution approach gets you surprisingly far and has the added benefit of being applicable on nearly all text-based configuration formats. XML is also fairly verbose, which also makes it unpopular in some circles. YML can be seen as a reaction to XML's verbosity and can accomplish many of the same things as XML, with less typing.

Another useful feature of some text configuration systems that deserves a mention is the idea that a base configuration file can include other configuration files. An example of this is the standard Unix sudo tool, which has its base configuration in the `/etc/sudoers` file, but which allows for local customization by including all the files that have been installed in the `/etc/sudoers.d` directory.

This is very convenient, because you can provide a new sudoer file without worrying too much about the existing configuration. This allows for a greater degree of modularization, and it is a convenient pattern when the application allows it.

Executing code on the client

Several of the configuration management systems described here allow you to reuse the node descriptors to execute code on matching nodes. This is sometimes convenient. For example, maybe you want to run a directory-listing command on all HTTP servers facing the public internet, perhaps for debugging purposes.

In the Puppet ecosystem, this command execution system is called **Marionette Collective**, or **MCollective** for short.

A note about the exercises

It is pretty easy to try out the various deployment systems using Docker to manage the base OS, where we will do our experiments. It is a time-saving method that can be used when developing and debugging the deployment code which is specific to a particular deployment system. This code will then be used for deployments on physical or virtual machines.

First, we will try each of the different deployment systems that are usually possible in local deployment modes. Further down the line, we will see how we can simulate the complete deployment of a system with several containers that together form a virtual cluster.

We will try to use the official Docker images if possible, but sometimes there are none, and sometimes the official image vanishes, which is what happened with the official Ansible image. Such is life in the fast-moving world of DevOps, for better or for worse.

It should be noted, however, that Docker has some limitations when it comes to emulating a full OS. Sometimes, a container must run in elevated privilege modes. We will deal with those issues when they arise.

It should also be noted that many people prefer Vagrant for these types of tests. I prefer to use Docker when possible, because it's lightweight, fast, and sufficient most of the time.

Keep in mind that actually deploying systems in production will require more attention to security and other details than we provide here.

The Puppet master and Puppet agents

Puppet is a deployment solution that is very popular in larger organizations and is one of the first systems of its kind.

Puppet consists of a client/server solution, where the client nodes check in regularly with the Puppet server to see if anything needs to be updated in the local configuration.

The Puppet server is called a **Puppet master**, and there is a lot of similar wordplay in the names chosen for the various Puppet components. Puppet provides a lot of flexibility in handling the complexity of a server farm, and, as such, the tool itself is pretty complex.

This is an example scenario of a dialogue between a Puppet client and a Puppet master:

- The Puppet client decides that it's time to check in with the Puppet master to discover any new configuration changes. This can be due to a timer or manual intervention by an operator at the client. The dialogue between the Puppet client and master is normally encrypted using SSL.
- The Puppet client presents its credentials so that the Puppet master knows exactly which client is calling. Managing the client's credentials is a separate issue.
- The Puppet master figures out which configuration the client should have by compiling the Puppet catalogue and sending it to the client. This involves a number of mechanisms, and a particular setup doesn't need to utilize all possibilities:
 - It is pretty common to have both a role-based configuration and a concrete configuration for a Puppet client. Role-based configurations can be inherited.

- The Puppet master runs the necessary code on the client side so that the configuration matches the one decided on by the Puppet master.

In this sense, a Puppet configuration is declarative. You declare what configuration a machine should have, and Puppet figures out how to get from the current to the desired client state.

There are both pros and cons of the Puppet ecosystem, and they are as follows:

- Puppet has a large community, and there are a lot of resources on the internet for Puppet. There are a lot of different modules, and if you don't have a really strange component to deploy, there already is, in all likelihood, an existing module written for your component that you can modify according to your needs.
- Puppet requires a number of dependencies on the Puppet client machines. Sometimes, this gives rise to problems. The Puppet agent will require a Ruby runtime that sometimes needs to be ahead of the Ruby version available in your distribution's repositories. Enterprise distributions often lag behind in versions.
- Puppet configurations can be complex to write and test.

Ansible

Ansible is a deployment solution that favors simplicity.

The Ansible architecture is agentless; it doesn't need a running daemon on the client-side like Puppet does. Instead, the Ansible server logs into the Ansible node and issues commands over SSH in order to install the required configuration.

While Ansible's agentless architecture does make things simpler, you need a Python interpreter installed on the Ansible nodes. Ansible is somewhat more lenient about the Python version required for its code to run than Puppet is for its Ruby code to run, so this dependence on Python being available is not a great hassle in practice.

Like Puppet and others, Ansible focuses on configuration descriptors that are idempotent. This basically means that the descriptors are declarative and the Ansible system figures out how to

bring the server to the desired state. You can rerun the configuration run, and it will be safe, which is not necessarily the case for an imperative system.

Let's try out Ansible with the Docker method we discussed earlier.

We will use the `williamyeh/ansible` image, which has been developed for this purpose, but it should be possible to use any Ansible Docker image or different ones altogether, to which we just add Ansible later:

1. Create a Dockerfile with this statement:

```
FROM williamyeh/ansible:centos7
```

2. Build the Docker container with the following command:

```
docker build .
```

This will download the image and create an empty Docker container that we can use. Normally, you would, of course, have a more complex Dockerfile that can add the things we need, but in this case, we are going to use the image interactively, so we will instead mount the directory with Ansible files from the host so that we can change them on the host and rerun them easily.

3. Run the container

The following command can be used to run the container. You will need the hash from the previous build command:

```
docker run -v `pwd`/ansible:/ansible -it <hash> bash
```

Now we have a prompt, and Ansible is available. The `-v` trick is to make parts of the host file system visible to the Docker guest container. The files will be visible in the `/ansible` directory in the container.

The `playbook.yml` file is as follows:

```
---  
- hosts: localhost  
  vars:  
    http_port: 80  
    max_clients: 200
```

```

remote_user: root

tasks:

- name: ensure apache is at the latest version
  yum: name=httpd state=latest

```

This playbook doesn't do very much, but it demonstrates some concepts of Ansible playbooks.

Now we can try to run our Ansible playbook:

```
cd /ansible
```

```
ansible-playbook -i inventory playbook.yml --connection=local
--sudo
```

The output will look like this:

```

                                PLAY                                [localhost]
*****

                                GATHERING                            FACTS
*****

    ok: [localhost]

TASK:  [ensure apache is at the latest version]
*****

    ok: [localhost]

                                PLAY                                RECAP
*****

****

localhost                                :  ok=2                changed=0
unreachable=0    failed=0

```

Tasks are run to ensure the state we want. In this case, we want to install Apache's httpd using yum, and we want httpd to be the latest version.

To proceed further with our exploration, we may like to do more things, such as start our services automatically. However, here, we run into a limitation with the approach of using Docker to emulate physical or virtual hosts. Docker is a container technology, after all, and not a full-blown virtualization system. In Docker's normal use case scenarios, this doesn't matter, but in our case, we need to make some workarounds in order to proceed. The main problem is that the systemd init system requires special care to run in a container. Developers at Red Hat have worked out methods of doing this. The following code is a slightly modified version of a Docker image by Vaclav Pavlin, who works with Red Hat:

```
FROM fedora

RUN yum -y update; yum clean all

RUN yum install ansible sudo

RUN          systemctl          mask          systemd-remount-fs.service
dev-hugepages.mount              sys-fs-fuse-connections.mount
systemd-logind.service getty.target console-getty.service

RUN          cp          /usr/lib/systemd/system/dbus.service
/etc/systemd/system/;      sed      -i      's/OOMScoreAdjust=-900//'
/etc/systemd/system/dbus.service

VOLUME ["/sys/fs/cgroup", "/run", "/tmp"]

ENV container=docker

CMD ["/usr/sbin/init"]
```

The environment variable container is used to tell the systemd init system that it runs inside a container and to behave accordingly.

We need some more arguments for docker run to enable systemd to work in the container:

```
docker run -it --rm -v /sys/fs/cgroup:/sys/fs/cgroup:ro -v
`pwd`/ansible:/ansible <hash>
```

The container boots with systemd, and now we need to connect to the running container from a different shell:

```
docker exec -it <hash> bash
```

Phew! That was quite a lot of work just to get the container to look more lifelike! On the other hand, working with VMs, such as VirtualBox, is even more cumbersome in my opinion. The reader may, of course, decide differently.

Now we can run a slightly more advanced Ansible playbook inside the container, as follows:

```
---
- hosts: localhost
  vars:
    http_port: 80
    max_clients: 200
    remote_user: root
  tasks:
    - name: ensure apache is at the latest version
      yum: name=httpd state=latest
    - name: write the apache config file
      template: src=/srv/httpd.j2 dest=/etc/httpd.conf
      notify:
        - restart apache
    - name: ensure apache is running (and enable it at boot)
      service: name=httpd state=started enabled=yes
  handlers:
    - name: restart apache
      service: name=httpd state=restarted
```

This code builds on the previous one, and shows you how to do the following:

- Install a package

- Write a template file
- Handle the running state of a service

The format is in a pretty simple YAML syntax.

Deploying with Chef

Chef is a Ruby-based deployment system from Opscode.

It is pretty easy to try out Chef; for fun, we can do it in a Docker container so we don't pollute our host environment with our experiments:

```
docker run -it ubuntu
```

We need the curl command to proceed with downloading the Chef installer:

```
apt-get -y install curl
```

```
curl -L https://www.opscode.com/chef/install.sh | bash
```

The Chef installer is built with a tool from the Chef team called **omnibus**. Our aim here is to try out a Chef tool called chef-solo. Verify that the tool is installed:

```
chef-solo -v
```

This will give us the following output:

```
Chef: 12.5.1
```

The point of chef-solo is to be able to run configuration scripts without the full infrastructure of the configuration system, such as the client/server setup. This type of testing environment is often useful when working with configuration systems, since it can be hard to get all the bits and pieces in working order while developing the configuration that you are going to deploy.

Chef prefers a file structure for its files, and a pre-rolled structure can be retrieved from GitHub.

You can download and extract it with the following commands:

```
curl -L http://github.com/opscode/chef-repo/tarball/master -o  
master.tgz
```

```
tar -zxvf master.tgz
```

```
mv chef-chef-repo* chef-repo
```

```
rm master.tgz
```

You will now have a suitable file structure prepared for Chef cookbooks, which looks like the following:

```
./cookbooks
```

```
./cookbooks/README.md
```

```
./data_bags
```

```
./data_bags/README.md
```

```
./environments
```

```
./environments/README.md
```

```
./README.md
```

```
./LICENSE
```

```
./roles
```

```
./roles/README.md
```

```
./chefignore
```

You will need to perform a further step to make everything work properly, telling chef where to find its cookbooks:

```
mkdir .chef
```

```
echo "cookbook_path [ '/root/chef-repo/cookbooks' ]" >  
.chef/knife.rb
```

Now we can use the knife tool to create a template for a configuration, as follows:

```
knife cookbook create phpapp
```

Deploying with SaltStack

SaltStack is a Python-based deployment solution.

There is a convenient dockerized test environment for Salt, by Jackson Cage. You can start it with the following code:

```
docker run -i -t --name=saltdocker_master_1 -h master -p 4505 -p
4506 \
-p 8080 -p 8081 -e SALT_NAME=master -e SALT_USE=master \
-v `pwd`/srv/salt:/srv/salt:rw jacksoncage/salt
```

This will create a single container with both a Salt master and a Salt minion.

We can create a shell inside the container for our further explorations:

```
docker exec -i -t saltdocker_master_1 bash
```

We need a configuration to apply to our server. Salt calls configurations states, or Salt states.

In our case, we want to install an Apache server with this simple Salt state:

```
top.sls:
```

```
base:
```

```
  '*':
```

```
    - webserver
```

```
webserver.sls:
```

```
apache2:                                     # ID declaration
```

```
pkg:                                         # state declaration
```

```
- installed                                # function declaration
```

Salt uses .yaml files for its configuration files, similar to what Ansible does.

The top.sls file declares that all matching nodes should be of the type webserver. The webserver state declares that an apache2 package should be installed, and that's basically it. Please note that this will be distribution dependent. The Salt Docker test image we are using is based on Ubuntu, where the Apache web server package is called apache2. On Fedora, for instance, the Apache web server package is simply called httpd, instead.

Run the command once to see Salt in action, by making Salt read the Salt state and apply it locally:

```
salt-call --local state.highstate -l debug
```

The first run will be very verbose, especially since we enabled the debug flag!

Now let's run the command again:

```
salt-call --local state.highstate -l debug
```

This will also be pretty verbose, and the output will end with this:

```
local:
```

```
-----
```

```
      ID: apache2
```

```
    Function: pkg.installed
```

```
      Result: True
```

```
    Comment: Package apache2 is already installed.
```

```
   Started: 22:55:36.937634
```

```
 Duration: 2267.167 ms
```

```
  Changes:
```

```
Summary
```

```
-----
```

```
Succeeded: 1
```

```
Failed:    0
```

```
-----
```

```
Total states run: 1
```

Now you can quit the container and restart it. This will clean the container from the Apache instance installed during the previous run.

This time, we will apply the same state but use the message queue method rather than applying the state locally:

```
salt-call state.highstate
```

This is the same command that we used previously, except we omitted the `-local` flag. You could also try running the command again and verify that the state remains the same.

Salt versus Ansible versus Puppet execution models

While the configuration systems we explored in this chapter share a fair number of similarities, they differ a lot in the way code is executed on the client nodes:

- With Puppet, a Puppet agent registers with the Puppet master and opens a communication channel to retrieve commands. This process is repeated periodically, normally every 30 minutes.

30 minutes isn't fast. You can, of course, configure a lower value for the time interval required for the next run. At any rate, Puppet essentially uses a pull model. Clients must check in to know whether changes are available.

- Ansible pushes changes over SSH when desired. This is a push model.
- Salt uses a push model, but with a different implementation. It employs a ZeroMQ messaging server that the clients connect to and listen for notifications about changes. This works a bit like Puppet, but faster.

Which method is best is an area of contention between developer communities. Proponents of the message queue architecture believe that it is faster and that speed matters. Proponents of the plain SSH method claim that it is fast enough and that simplicity matters. I lean towards the latter stance. Things tend to break, and the likelihood of breakage increases with complexity.

Deploying with Docker

A recent alternative for deployment is Docker, which has several very interesting traits. We have already used Docker several times in this book. You can make use of Docker's features for test automation purposes even if you use, for instance, Puppet or Ansible to deploy your products. Docker's model of creating reusable containers that can be used on development machines, testing environments, and production environments is very appealing.

Docker is beginning to have an impact on larger enterprises, but solutions such as Puppet or Ansible are still dominant. While it is well-known how to build large Puppet or Ansible server farms, it's not yet equally well-known how to build large Docker-based server clusters.

There are several competing solutions, such as these:

- **Docker swarm:** Docker swarm is compatible with Docker Compose, which is appealing. Docker Swarm is maintained by the Docker community.
- **Kubernetes:** Kubernetes is modeled after Google's Borg cluster software, which is appealing since it's a well-tested model used in-house in Google's vast data centers. Kubernetes is not the same as Borg, though, which must be kept in mind. It's not clear whether Kubernetes offers scaling the same way Borg does.

There are also orchestration systems running mostly on top of Kubernetes, such as OpenShift. A serious on-premise installation would require systems such as these.

Comparison tables

Everyone likes coming up with new words for old concepts. While the different concepts in various products don't always match, it's tempting to make a dictionary that maps the configuration systems' different terminologies with one another.

Here is such a terminology comparison table:

System	Puppet	Ansible	Salt
Client	Agent	Node	Minion
Server	Master	Server	Master
Configuration	Catalog	Playbook	Salt State

Also, here is a technology comparison table:

System	Puppet	Ansible	Chef	Salt
Agentless	No	Yes	Yes	Both
Client dependencies	Ruby	Python, SSHD, bash	Ruby, SSHD, bash	Python
Language	Ruby	Python	Ruby	Python

Cloud solutions

First, we must take a step back and have a look at the landscape. We can either use a cloud provider, such as AWS or Azure, or we can use our own internal cloud solution, such as VMware

or OpenStack. There are valid arguments for both external and internal cloud providers, or even both, depending on your organization.

Some types of organizations, such as government agencies, must store all data regarding citizens within their own walls. Such organizations can't use external cloud providers and services, and must instead build their own internal cloud equivalents.

Smaller private organizations may benefit from using an external cloud provider, but can't perhaps afford having all of their resources with such a provider. They may opt to have in-house servers for normal loads and scale out to an external cloud provider during peak loads.

Many of the configuration systems we have described here support the management of cloud nodes as well as local nodes. PalletOps supports AWS and Puppet supports Azure, for instance. Ansible supports a host of different cloud services.

AWS

AWS allows us to deploy VM images on Amazon's clusters. You can also deploy Docker images.

Follow these steps to set up AWS:

1. Sign up for an account with AWS. Registration is free of charge, but a credit card number is required, even for the free of charge tier.
2. Some identity verification will need to happen, which can be done via an automated challenge-response phone call.
3. When the user verification process is complete, you will be able to log into AWS and use the web console.

In my opinion, the AWS web console does not represent the epitome of web interface usability, but it gets the job done. There are a host of options, and, in our case, we are interested in the VM and Docker container options.

4. Go to EC2 network and security. Here, you can create management keys that will be required later.

5. As a first example, let's create the default container example provided by AWS, `console-sample-app-static`. To log in to the generated server, you need first to create an SSH key pair and upload your public key to AWS. Click through all the steps and you will get a small sample cluster. The final resource creation step can be slow, so it's the perfect opportunity to grab a cup of coffee!
6. Now we can view the details of the cluster and choose the web server container. You can see the IP address. Try opening it in a web browser.

Now that we have a working account on AWS, we can manage it with the configuration management system of our choice.

Azure

Azure is a cloud platform from Microsoft. It can host both Linux and Microsoft VMs. While AWS is the service people often default to, at least in the Linux space, it never hurts to explore different options. Azure is one such option that is gaining market share at the moment.

Creating a VM on Azure for evaluation purposes is comparable to creating a VM on AWS. The process is fairly smooth.

Monitoring the Code

In the previous chapter, [Chapter 7](#), *Deploying the Code*, we explored methods we could use for deploying our code.

Now that the code has been safely deployed to your servers with the deployment solution of your choice, you need to watch over it to make sure it's running properly. You can spend a lot of time preparing for the many modes of failure that you may envision during development. In the end, your software will probably break for other reasons altogether than those you prepared for. If

your system breaks, it can be very expensive for your organization, either in lost revenue or in terms of lost credibility, which, in the end, might amount to the same thing. You need to know as quickly as possible what has gone wrong in order to deal with the situation.

Given the potential negative impact of service downtime, there are many alternative solutions that approach the problem domain of watching over your deployed code from different angles and viewpoints.

In this chapter, we will have a look at several options that are available to us.

Nagios

In this section, we will explore the Nagios monitoring solution for overall server health.

Nagios has been around since 1999 and has a large community that provides plugins and extensions for various needs. There are extensive resources available for Nagios on the web, and commercial support if your organization requires it.

Since it has been around so long and many organizations use it, Nagios is the standard in network monitoring, which other solutions compare themselves with. As such, it is the natural starting point for our journey through the monitoring landscape.

The name Nagios is a recursive acronym, which is a tradition in hacker circles. It stands for "*Nagios Ain't Gonna Insist On Sainthood*." Originally, the product was called NetSaint, but the name was rejected in favor of the Nagios acronym during a trademark dispute. The word *agios* also means *angel* in Greek, so it is a pretty clever acronym overall.

Here are some screenshots from a small demo Nagios installation. Nagios provides many views, and here, we can see two of them:

- Service Overview For All Host Groups:

General

[Home](#)
[Documentation](#)

Current Status

[Tactical Overview](#)
[Map](#)
[Hosts](#)
[Services](#)
[Host Groups](#)

[Summary](#)
[Grid](#)

[Service Groups](#)

[Summary](#)
[Grid](#)

[Problems](#)

[Services \(Unhandled\)](#)
[Hosts \(Unhandled\)](#)
[Network Outages](#)

Quick Search:

Reports

[Availability](#)
[Trends](#)
[Alerts](#)

[History](#)
[Summary](#)
[Histogram](#)

[Notifications](#)
[Event Log](#)

System

[Comments](#)
[Downtime](#)
[Process Info](#)
[Performance Info](#)
[Scheduling Queue](#)
[Configuration](#)

Current Network Status

Last Updated: Wed Nov 11 00:09:11 CET 2015
Updated every 90 seconds
Nagios® Core™ 3.5.1 - [www.nagios.org](#)
Logged in as *joakim*

[View Service Status Detail For All Host Groups](#)
[View Host Status Detail For All Host Groups](#)
[View Status Summary For All Host Groups](#)
[View Status Grid For All Host Groups](#)

Host Status Totals

Up	Down	Unreachable	Pending
6	1	0	0
All Problems		All Types	
1		7	

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
17	1	3	13	0
All Problems		All Types		
17		34		

Service Overview For All Host Groups

Linux Servers (linux-servers)

Host	Status	Services	Actions
localhost	UP	11 OK 5 CRITICAL	

Network Printers (network-printers)

Host	Status	Services	Actions
hpcolor	UP	1 OK 1 WARNING	

Network Switches (switches)

Host	Status	Services	Actions
netgear-prosafe	UP	1 OK 3 UNKNOWN	

Verona Linux Servers (verona-servers)

Host	Status	Services	Actions
exodia	UP	No matching services	
gate	UP	No matching services	
gw	UP	4 OK 2 CRITICAL	
localhost	UP	11 OK 5 CRITICAL	
mentor-xbmc	DOWN	6 CRITICAL	

- Service Status Details For All Hosts:

Nagios®

Current Network Status
 Last Updated: Wed Nov 11 00:10:44 CET 2015
 Updated every 90 seconds
 Nagios® Core™ 3.5.1 - www.nagios.org
 Logged in as joakim

Host Status Totals

Up	Down	Unreachable	Pending
6	1	0	0

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
17	1	3	13	0

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
gw	/dev/hda1	CRITICAL	11-11-2015 00:04:18	1271d 7h 6m 49s	3/3	DISK CRITICAL - /dev/hda1 is not accessible: No such file or directory
	Free Space	OK	11-11-2015 00:08:18	135d 12h 53m 19s	1/3	OK - load average: 0.00, 0.01, 0.05
	CPU Load	OK	11-11-2015 00:09:18	135d 12h 54m 19s	1/3	USERS OK - 3 users currently logged in
	Current Users	OK	11-11-2015 00:03:18	135d 12h 54m 19s	1/3	PROCS OK: 110 processes
	Total Processes	OK	11-11-2015 00:02:18	135d 12h 51m 19s	1/3	PROCS OK: 0 processes with STATE = Z
	Zombie Processes	OK	11-11-2015 00:04:18	1271d 7h 2m 53s	3/3	NRPE: Command 'check_updates' not defined
hpcolor	updates	CRITICAL	11-11-2015 00:09:18	1d 3h 1m 26s	1/3	PING OK - Packet loss = 0%, RTA = 0.30 ms
	PING	OK	11-11-2015 00:08:18	3d 13h 12m 26s	3/3	Out of Paper ("Fyll p. papper.")
localhost	Printer Status	WARNING	11-11-2015 00:09:18	5d 1h 21m 26s	1/4	OK - load average: 0.70, 0.72, 0.67
	Current Load	OK	11-11-2015 00:07:18	1441d 9h 7m 32s	1/4	USERS OK - 4 users currently logged in
	Current Users	OK	11-11-2015 00:07:18	226d 0h 55m 14s	1/4	HTTP OK: HTTP/1.1 200 OK - 3933 bytes in 0.000 second response time
	HTTP	OK	11-11-2015 00:08:18	399d 2h 8m 56s	4/4	PROCS CRITICAL: 0 processes with command name 'mynet'
	Mldonkey	CRITICAL	11-11-2015 00:09:18	739d 19h 29m 22s	1/4	PING OK - Packet loss = 0%, RTA = 0.05 ms
	PING	OK	11-11-2015 00:08:18	56d 13h 3m 11s	4/4	DISK CRITICAL - free space: / 6200 MB (4% inode=85%):
	Root Partition	CRITICAL	11-11-2015 00:08:18	739d 19h 27m 49s	1/4	SSH OK - OpenSSH_6.9 (protocol 2.0)
	SSH	OK	11-11-2015 00:09:18	1441d 9h 4m 25s	1/4	SWAP OK - 97% free (7574 MB out of 7887 MB)
	Swap Usage	OK	11-11-2015 00:08:18	30d 0h 43m 18s	4/4	PROCS CRITICAL: 726 processes with STATE = RSZDT
	Total Processes	CRITICAL	11-11-2015 00:09:18	523d 17h 5m 22s	1/4	• bitlbee.service - BitBee IRC/IM gateway
	bitlbee	OK	11-11-2015 00:07:18	226d 5h 30m 23s	1/4	• dovecot.service - Dovecot IMAP/POP3 email server
	dovecot	OK	11-11-2015 00:08:18	305d 1h 48m 7s	1/4	• fetchmail.service - SYSV: Fetchmail mail retrieval program
	fetchmail	OK	11-11-2015 00:08:18	465d 7h 50m 31s	4/4	connect to address localhost and port 25565: Connection refused
	minecraft	CRITICAL	11-11-2015 00:08:18	1440d 21h 3m 43s	1/4	127.0.0.1 10 20
	test	OK	11-11-2015 00:08:18			CHECK_UPDATES CRITICAL - your machine is running kernel 4.1.10-200

The views indicate some example problems, as follows:

- A host is unreachable
- A disk in one of the hosts is getting full

A host has pending updates available. Let's examine the basics of Nagios in particular and monitoring in general with a simple scenario:

- We have a Nagios host that monitors the well-being of a number of other physical and virtual hosts.

- One of the hosts being monitored, a web server, suddenly vanishes from the network altogether. Nagios knows this, since it is set up to ping the web server every five minutes and take action in the event that the server does not respond to the pings in a timely fashion.
- Nagios determines what to do when this server is down, which in this case is to send an email to everyone in a predetermined group (in this case, the administrators of the organization).
- Joakim, who happens to be around the Matangle office, is the first to respond to the email, and goes down to the quite unprofessional server room in the cellar. The web server has stopped working because its power supply unit is full of dust, which accumulated there while renovations were being done in a nearby room—sawing plaster generates a lot of dust. Joakim curses and vows that he would move the servers to better hosting, if he only had the resources.

You likely take better care of your servers than Joakim did in this scenario, but the point is that unforeseen things happen, and Nagios was helpful in resolving the situation. Also, while employing a proper data center for your servers might prevent things such as dust in your equipment and people tripping over network cables, there are still many weird and unforeseen modes of failure present in the unforgiving world of complex server deployments.

Nagios can monitor services as well as hosts and can utilize different methods to do so. The simplest methods work on the Nagios server and query a service on a remote host.

Nagios differentiates between two basic classes of checks:

- **Active checks:** These are initiated by Nagios. Nagios can execute code in plugins regularly. The outcome of the code execution determines whether the check fails or not. Active checks are the most common and are useful and easy to set up with many types of services, such as HTTP, SSH, and databases.
- **Passive checks:** These originate from a system other than Nagios, and the system notifies Nagios rather than Nagios polling it. These are useful in special situations, for

instance, if the service being monitored is behind a firewall or if the service is asynchronous and it would be inefficient to monitor it with polling.

Configuring Nagios is relatively straightforward if you are used to file-based configuration, but there is a lot of configuring to do in order to get basic functionality.

We are going to explore Nagios with the Docker Hub image `jasonrivers/nagios`.

This statement starts a Nagios server container:

```
sudo docker run -e NAGIOSADMIN_USER=nagiosadmin -e  
NAGIOSADMIN_PASS=nagios -p 30000:80 jasonrivers/nagios
```

Verify that the Nagios web interface is available on port 30000; enter the username and password defined previously to log in.

The out-of-the-box configuration of the Nagios user interface allows you to browse, among other things, hosts and services, but the only host defined at the outset is the localhost, which in this case is the container running Nagios itself. We will now configure another host to monitor, in the form of a separate Docker container. This will allow us to bring the monitored container down by force and verify that the Nagios container notices the outage.

This command will start a second container running `nginx` in the default configuration:

```
docker run -p 30001:80 nginx
```

Verify that the container is running by going to port 30001. You will see a simple message, Welcome to nginx!. This is more or less all that this container does out of the box, which is good enough for our test. You can, of course, use a physical host instead if you prefer.

To run the containers together, we can either link them on the command line or use a Docker compose file. Here is a Docker compose file for this scenario, which is also available in the book sources:

```
nagios:  
  image: mt-nagios  
  build:  
    - mt-nagios
```

```

ports:
  - 80:30000

environment:
  - NAGIOSADMIN_USER=nagiosadmin
  - NAGIOSADMIN_PASS=nagios

volumes:
  ./nagios:/etc/nagios

nginx:
  image: nginx

```

The Nagios configuration is mounted in the `./nagios` directory as a Docker volume.

The configuration required for Nagios to monitor the nginx container is available in the sources and is also included here:

```

define host {
    name          regular-host
    use           linux-server
    register      0
    max_check_attempts 5
}

define host{
    use          regular-host
    host_name    client1
    address      192.168.200.15
    contact_groups admins
    notes        test client1
}

hostgroups.cfg

```

```
define hostgroup {  
    hostgroup_name test-group  
    alias          Test Servers  
    members        client1  
}
```

services.cfg

```
#+BEGIN_SRC sh
```

```
define service {  
    use                generic-service  
    hostgroup_name     test-group  
    service_description PING  
    check_command       check_ping!200.0,20%!600.0,60%  
}
```

Let's see what happens when we bring the nginx container down.

Find out the hash of the nginx container with `docker ps`. Then, kill it with `docker kill`. Verify that the container is really gone with `docker ps` again.

Now, wait a while and reload the Nagios web user interface. You should see that Nagios has alerted you of the situation. This will look similar to the earlier screenshots that showed faulty services.

Now, you would like to receive an email when nginx goes down. It is, however, not straightforward to make an example that works in a foolproof way, simply since email is more complicated these days due to spam. You need to know your mail server details, such as the **Simple Mail Transfer Protocol (SMTP)** server details. Here is a skeleton you need to fill out with your particular details:

You can create a `contacts.cfg` file for handling emails, with the following contents:

```
define contact{
```

```
    contact_name      matangle-admin
    use               generic-contact
    alias            Nagios Admin
    email            pd-admin@matangle.com
}
```

```
define contactgroup{
    contactgroup_name  admins
    alias             Nagios Administrators
    members           matange-admin
}
```

If you don't change this configuration, the mail will wind up at pd-admin@matangle.com, which will be ignored.

Munin

Munin is used to graph server statistics such as memory usage, which is useful in order to understand overall server health. Since Munin graphs statistics over time, you can see resource allocation trends, which can help you find problems before they get serious. There are several other applications like Munin that create graphs; however, like Nagios, Munin is a good starting point.

It is designed to be easy to use and set up. The out-of-the-box experience gives you many graphs with little work.

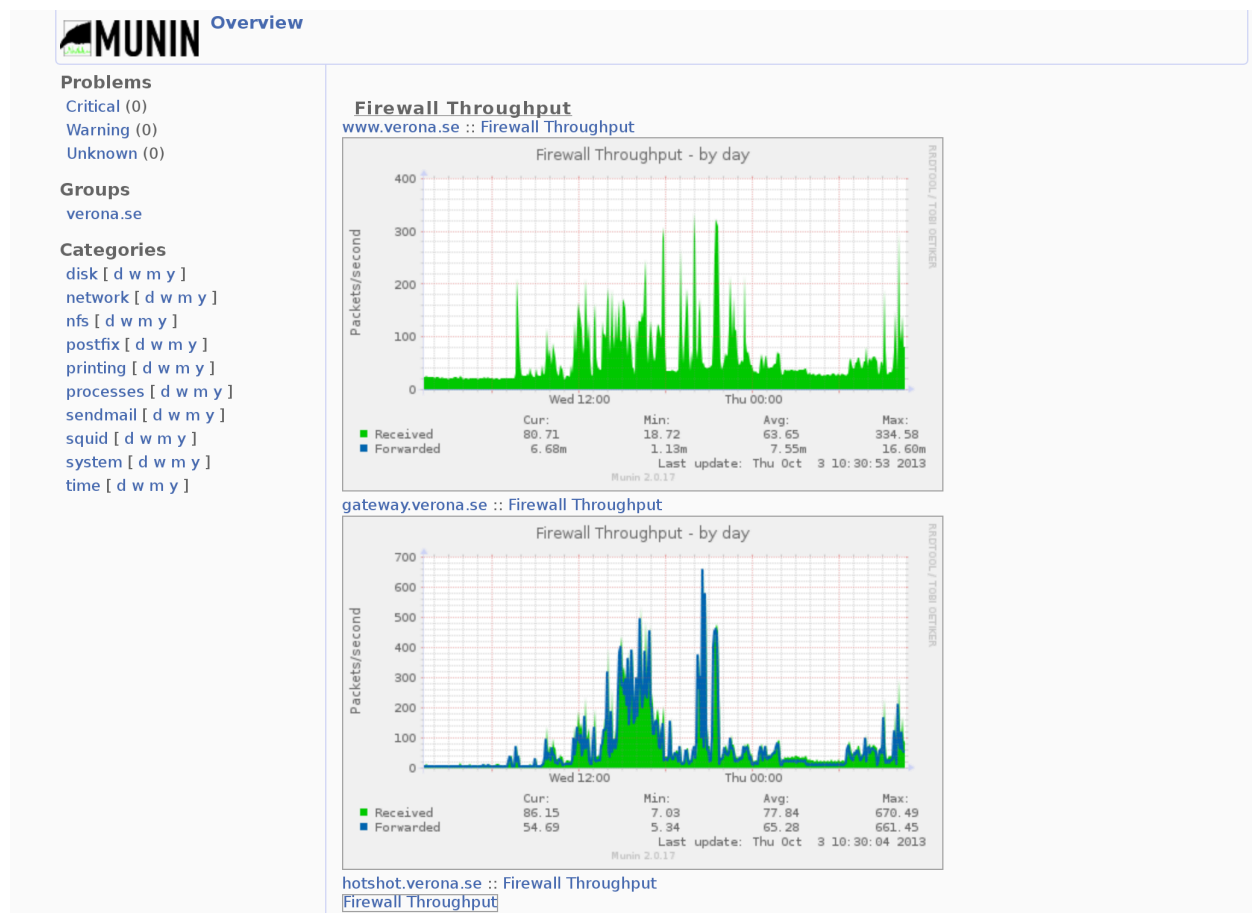
In the legends of the Norse, Hugin and Munin were two pitch-black ravens. They tirelessly flew around the world of Midgard and collected information. After their journeys far and wide, they returned to the god king Odin to sit on his shoulders and tell him all about their experiences. The name Hugin derives from the word for thought and Munin from the word for memory.

While Nagios focuses on the high-level traits of the health of a service (whether the service or host is alive or not in binary terms), Munin keeps track of statistics that it periodically samples, and draws graphs of them.

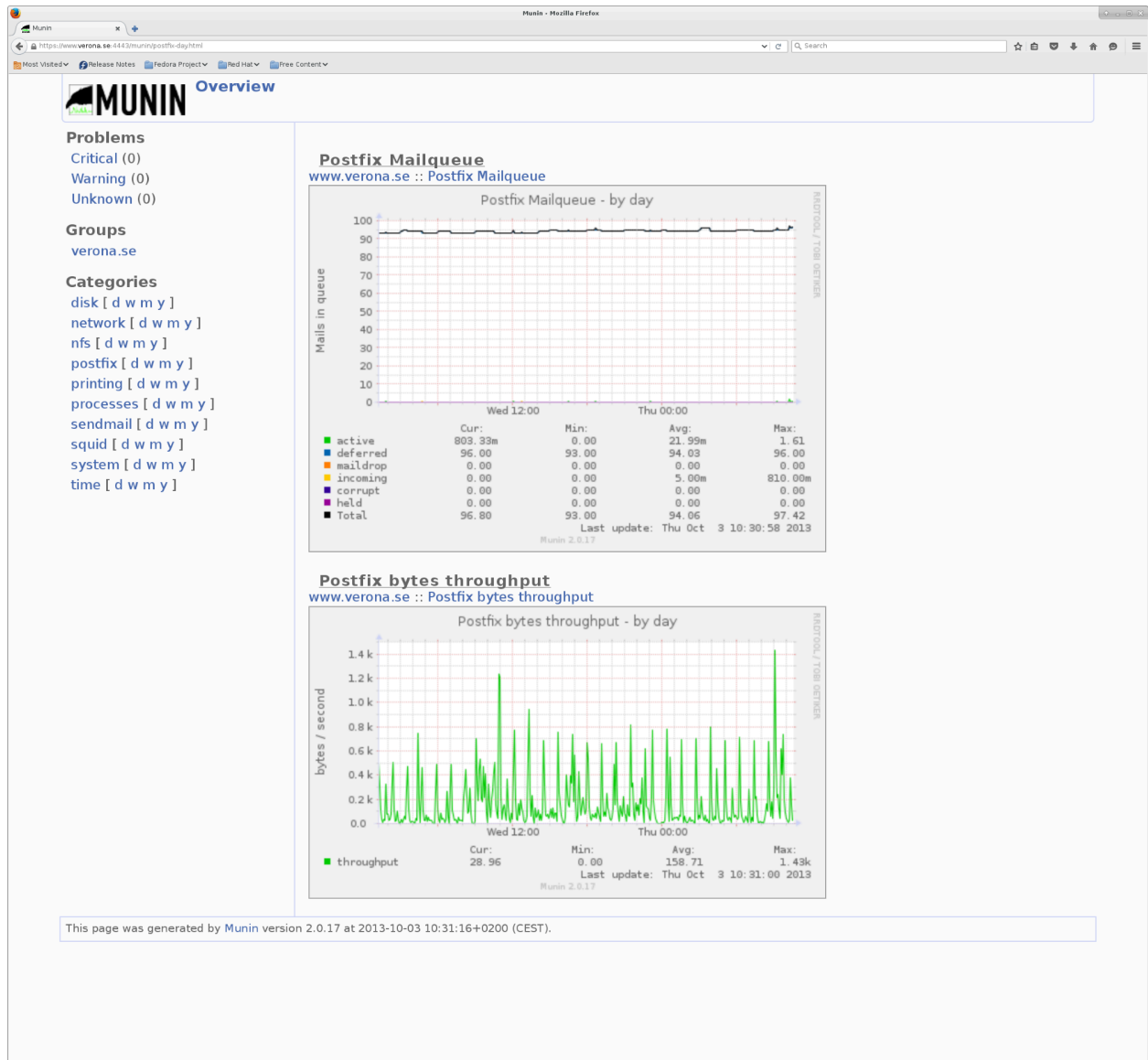
Munin can sample a lot of different types of statistics, from CPU and memory load to the number of active users in your children's Minecraft server. It is also easy to make plugins that get you the statistics you want from your own services.

An image can say a lot and convey lots of information at a glance, so graphing the memory and processor load of your servers can give you an early warning if something is about to go wrong.

Here is an example screenshot of Munin, displaying metrics for a firewall installation at the Matangle headquarters:



And here are some metrics for the organization's smtpd, a Postfix installation:



Like any of the monitoring systems we are exploring in this chapter, Munin also has a network-oriented architecture, as explained here. It is similar in design to Nagios. The main components of Munin are as follows:

- There is a central server, the Munin master, which is responsible for gathering data from the Munin nodes. The Munin data is stored in a database system called a **round-robin database (RRD)**. The RRD also does the graphing of the gathered data.

- The Munin node is a component that is installed on the servers that will be monitored. The Munin master connects to all the Munin nodes and runs plugins that return data to the master.

To try it out, we will once again use a Docker container running the service we are exploring, Munin:

```
docker run -p 30005:80 lrivallain/munin:latest
```

Munin will take a while to run the first time, so wait a while before checking the web interface. If you don't like to wait, you can run the munin-update command by hand in the container, as shown here. It polls all the Munin nodes for statistics explicitly:

```
docker run exec -it <hash> bash
```

```
su - munin --shell=/bin/bash
```

```
/usr/share/munin/munin-update
```

Now, you should be able to see the graphs created during the first update. If you let the stack run for a while, you can see how the graphs develop.

It is not very hard to write a Munin plugin that monitors a statistic that is specific to your application stack. You can make a shell script that Munin calls in order to get the statistics you want to track.

Munin itself is written in Perl, but you can write Munin plugins in most languages as long as you conform to the very simple interface.

The program should return some metadata when called with a config argument. This is so that Munin can put proper labels on the graphs.

Here is an example graph configuration:

```
graph_title Load average
```

```
graph_vlabel load
```

```
load.label load
```

To emit data, you simply print it to stdout:

```
printf "load.value "
```

```
cut -d' ' -f2 /proc/loadavg
```

Here is an example script that will graph the machine's load average:

```
#!/bin/sh
```

```
case $1 in
```

```
config)
```

```
cat <<'EOM'
```

```
graph_title Load average
```

```
graph_vlabel load
```

```
load.label load
```

```
EOM
```

```
exit 0;;
```

```
esac
```

```
printf "load.value "
```

```
cut -d' ' -f2 /proc/loadavg
```

This system is pretty simple and reliable, and you can probably easily implement it in your application. All you need to do is to be able to print your statistics to stdout.

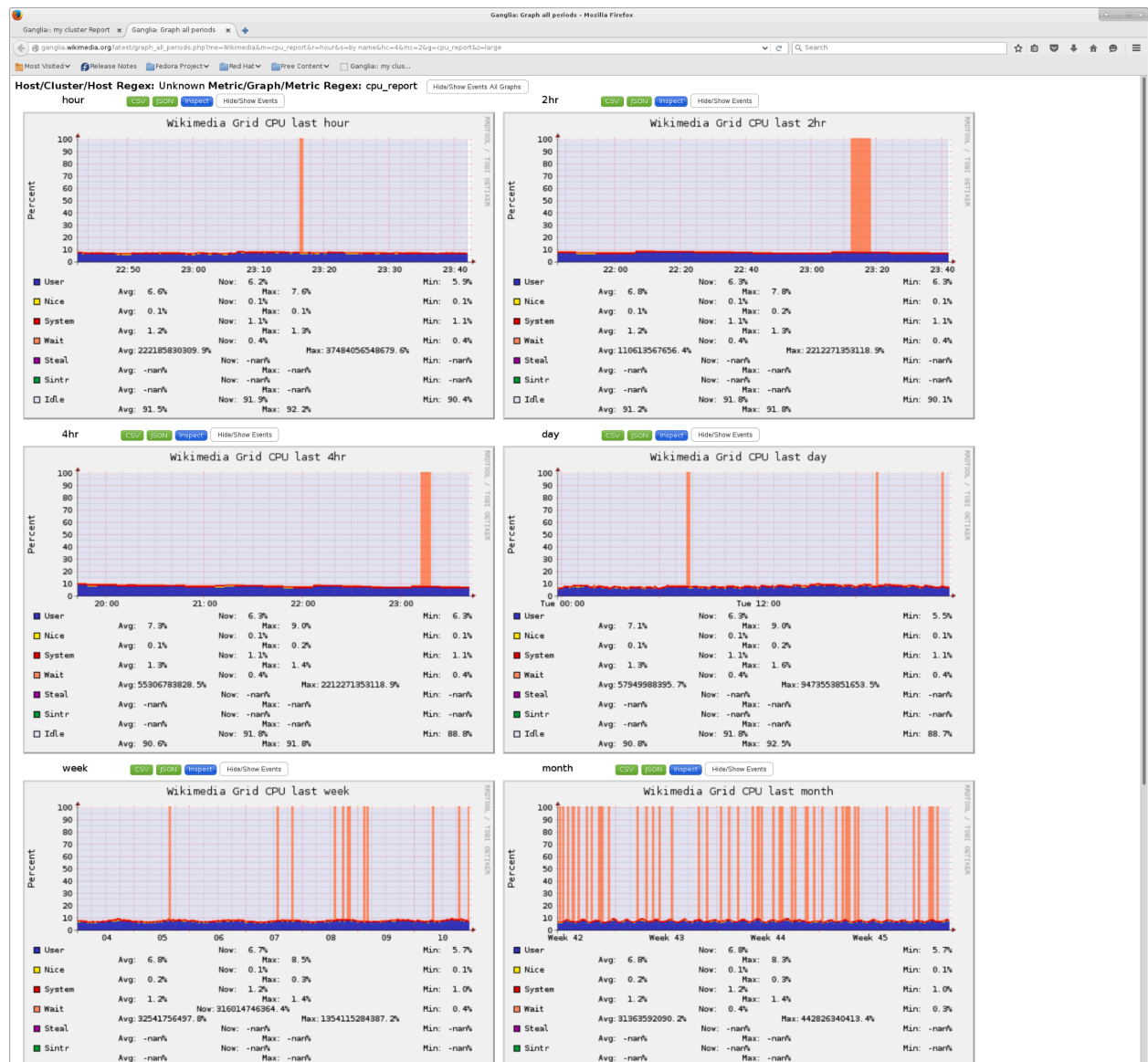
Ganglia

Ganglia is a graphing and monitoring solution for large clusters. It can aggregate information in convenient overview displays.

The word *ganglia* is the plural form of ganglion, which is a nerve cell cluster in anatomy. The analogy implies that Ganglia can be the sensory nerve cell network in your cluster.

Like Munin, Ganglia also uses RRDs for database storage and graphing, so the graphs will look similar to the previous Munin graphs. Code reuse is a good thing!

Ganglia's start page shows an overview of the available data in graph format. You can drill down in the graphs to get other perspectives:



If you drill down one of the application cluster graphs for CPU load, for example, you get a detailed view of the individual servers in the cluster. You can click on a server node to drill down further.

If you have a large cluster, you can clearly see the attraction of Ganglia overviews and drill-down views. You have both a view from orbit and the details easily accessible together.

Ganglia consists of the following components:

- **Gmond:** This is an acronym for **Ganglia monitoring daemon**. Gmond is a service that collects information about a node. Gmond will need to be installed on each server that you want Ganglia to monitor.
- **Gmetad:** This stands for **Ganglia meta daemon**. Gmetad is a daemon that runs on the master node, collecting the information that all the Gmond nodes gather. Gmetad daemons can also work together to spread the load across the network. If you have a large enough cluster, the topology does indeed start to look like a nerve cell network!
- **RRD:** An RRD, the same tool Munin uses on the master node to store data and visualizations for Ganglia in a time series that is suitable for graphing.
- **A PHP-based web frontend:** This displays the data that the master node has collected and RRD has graphed for us.

Compared to Munin, Ganglia has an extra layer, the meta daemon. This extra layer allows Ganglia to scale by distributing the network load between nodes.

Ganglia has a grid concept, where you group clusters with a similar purpose together. You can, for example, put all your database servers together in a grid. The database servers don't need to have any other relationship; they can serve different applications. The grid concept allows you to view the performance metrics of all your database servers together.

You can define grids in any manner you like: it can be convenient to view servers by location, by application, or by any other logical grouping you need.

You can try Ganglia out locally as well, again, using a Docker image.

The wookietreiber/ganglia Docker image from Docker Hub offers some inbuilt help:

```
docker run wookietreiber/ganglia --help
```

```
Usage: docker run wookietreiber/ganglia [opts]
```

```
Run a ganglia-web container.
```

<code>-? -h -help --help</code>	<code>print this help</code>
<code>--with-gmond</code>	<code>also run gmond inside the container</code>
<code>--without-gmond</code>	<code>do not run gmond inside the container</code>
<code>--timezone arg</code>	<code>set timezone within the container,</code>
	<code>must be path below /usr/share/zoneinfo,</code>
	<code>e.g. Europe/Berlin</code>

In our case, we will run the image with the following command-line arguments:

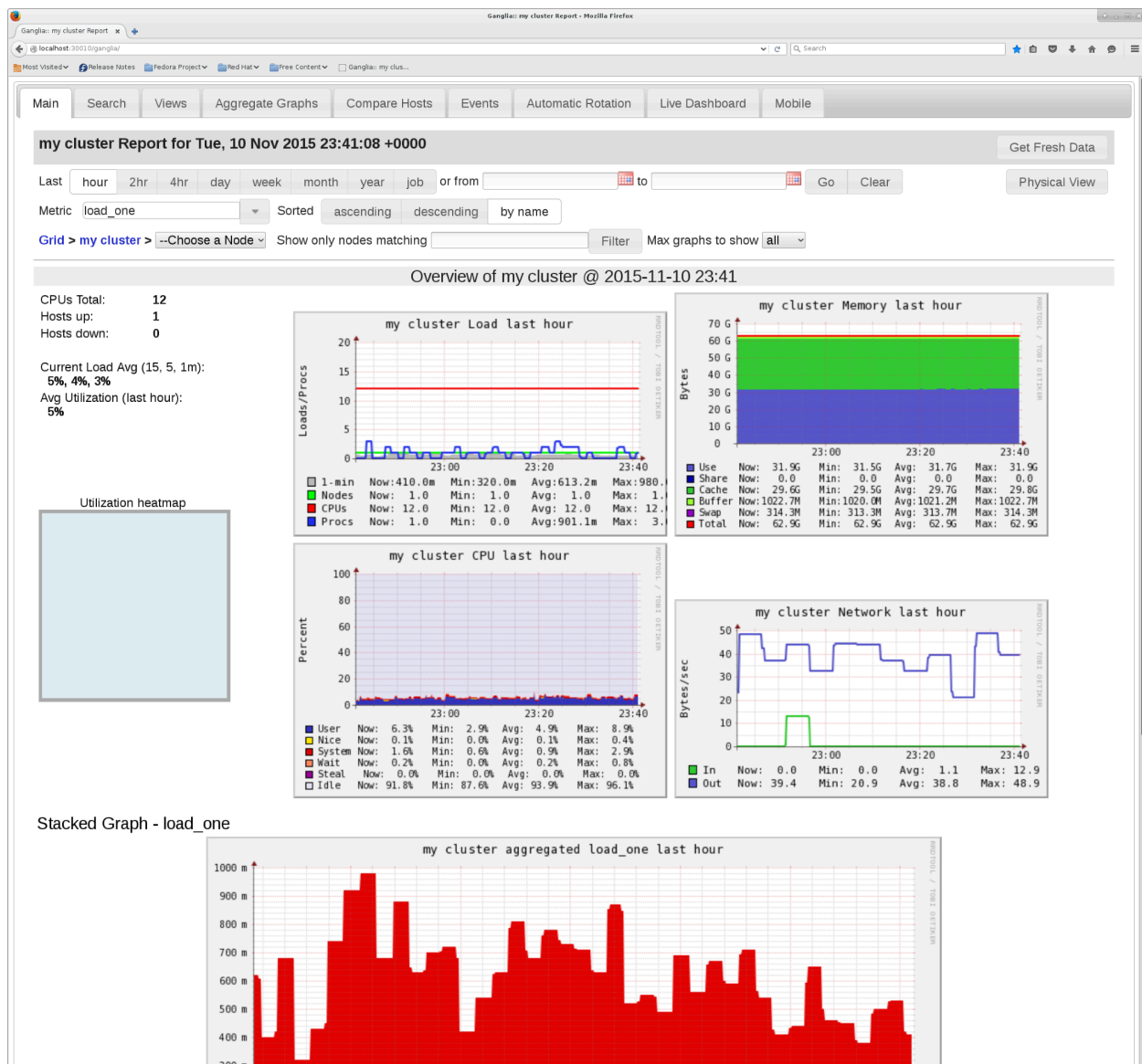
```
docker run -p 30010:80 wookietreiber/ganglia
```

This image runs Gmond as well as Gmetad. This means that it will monitor itself out of the box.

We will add a separate container running Gmetad in a moment.

Now that the Ganglia container is running, you can check out the web interface served by the container: <http://localhost:30010/ganglia/>

The browser should show a view similar to the following screenshot:



Ganglia Gmond nodes communicate between themselves on a multicast IP channel. This offers redundancy and ease of configuration in large clusters, and the multicast configuration is default. You can also configure Ganglia in unicast mode.

Since we will only run with two communicating containers, we will not actually benefit from the multicast configuration in this case.

Again, we will use Docker Compose to start the containers together so that the separate gmond and gmetad instances can communicate. We can start three containers that all run gmond. The

gmetad daemon will, in the default configuration, discover them and list them together under My Cluster:

```
gmetad:
  image: wookietreiber/ganglia
  ports:
    - "30010:80"
gmond:
  image: wookietreiber/ganglia
gmond2:
  image: wookietreiber/ganglia
```

When you access <http://localhost:30010/ganglia/>, you should find the new gmond instance being monitored.

Graphite

While Munin is nice because it is robust and fairly easy to start using, the graphs it provides are only updated once in a while, normally every fifth minute. There is, therefore, a niche for a tool that does graphing that is closer to real time. Graphite is such a tool.

The Graphite stack consists of the following three major parts. It is similar to both Ganglia and Munin, but uses its own component implementations:

- The Graphite web component, which is a web application that renders a user interface consisting of graphs and dashboards organized within a tree-like browser widget
- The Carbon metric processing daemon, which gathers the metrics
- The Whisper time series database library

As such, the Graphite stack is similar in utility to both Munin and Ganglia. Unlike Munin and Ganglia though, it uses its own time series library, Whisper, rather than an RRD.

There are several prepackaged Docker images for trying out Graphite. We can use the sitespeedio/graphite image from the Docker Hub, as follows:

```
docker run -it \ -p 30020:80 \ -p 2003:2003 \ sitespeedio/graphite
```

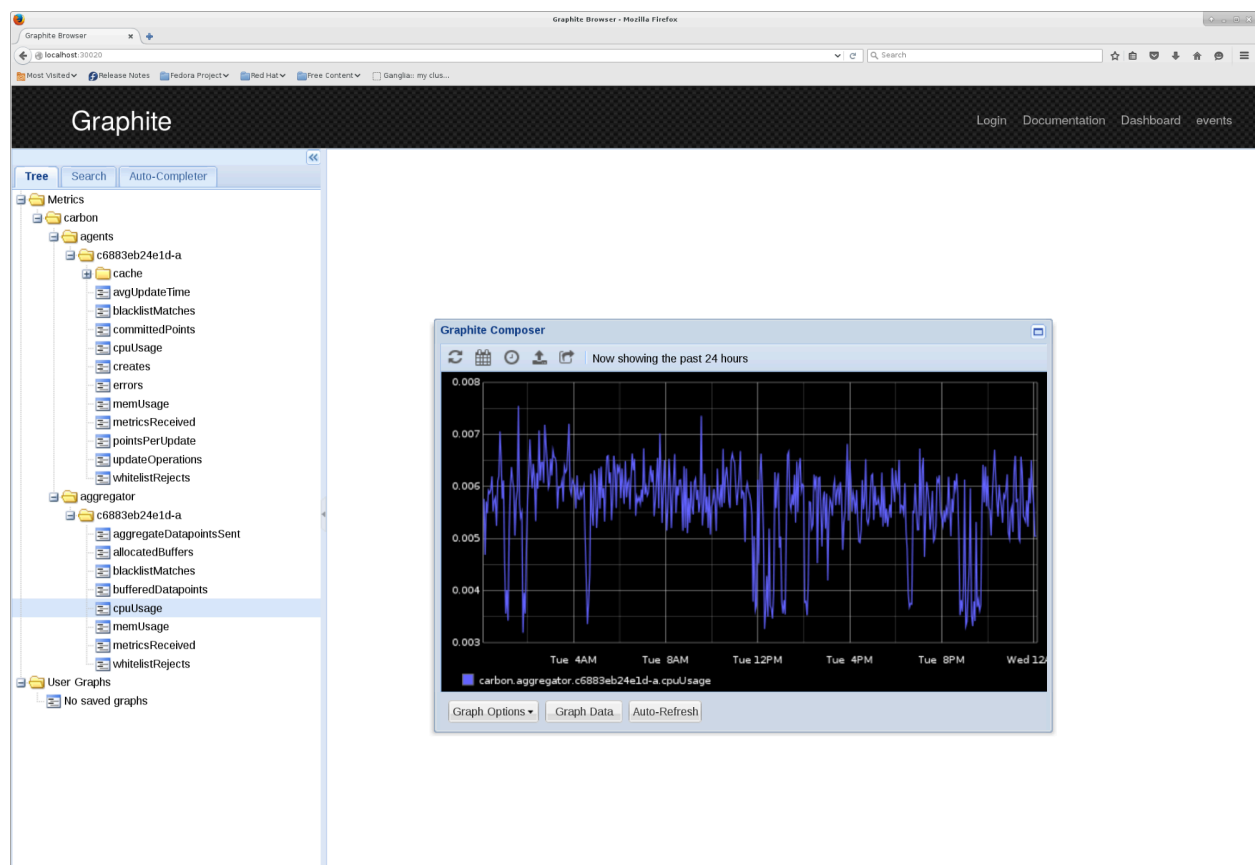
This starts a Docker container running Graphite with HTTP basic authentication.

You can now view the user interface of Graphite. Enter guest for both the username and password. You can change the username and password by providing a .htpasswd file for the image.

If you don't change the port in the preceding Docker statement, this URL should work:

<http://localhost:30020/>

The browser window should display a view similar to the following screenshot:



Graphite has a customizable user interface, where you can organize the graphs that you are interested in together in dashboards. There is also a completion widget that lets you find named graphs by typing the first few characters of a graph name.

Log handling

Log handling is a very important concept, and we will explore some of the many options, such as the **Elasticsearch**, **Logstash**, and **Kibana (ELK)** stack.

Traditionally, logging just consisted of using simple print statements in code to trace events in the code. This is sometimes called **printf-style debugging**, because you use traces to see how your code behaves rather than using a regular debugger.

Here is a simple example in C syntax. The idea is that we want to know when we can enter the function $fn(x)$ and what value the argument x has:

```
void fn(char *x){  
    printf("DEBUG entering fn, x is %s\n", x);  
    ...  
}
```

From the debug traces in the console, you can determine whether the program being developed is behaving as expected.

You would, of course, also like to see whether something serious is wrong with your program and report that with a higher priority:

```
printf("ERROR x cant be an empty string\n");
```

There are several problems with this style of debugging. They are useful when you want to know how the program behaves, but they are not so useful when you have finished developing and want to deploy your code.

Today, there are a great number of frameworks that support logging in many different ways, based on the time-proven pattern we have just seen.

The logging frameworks add value to printf style logging primarily by defining standards and offering improved functionality such as these:

- Different log priorities, such as debug, warning, trace, and error.
- Filtering log messages of different priorities. You might not always be interested in debug traces, but you are probably always interested in error messages.
- Logging to different destinations, such as files, databases, or network daemons. This includes the ELK stack that we will visit later.
- The rotating and archiving of log files. Old log files can be archived.

Every now and then, a new logging framework pops up, so the logging problem domain seems far from exhausted, even to this day. This tendency is understandable, since properly done logs can help you determine the exact cause of the failure of a network service that is no longer running or, of course, any complicated service that you are not constantly supervising. Logging is also hard to do right, since excessive logging can kill the performance of your service, and too little doesn't help you determine the cause of failures. Therefore, logging systems go to great lengths to strike a balance between the various traits of logging.

Client-side logging libraries

Log4j is a popular logging framework for Java. There are several ports for other languages, such as:

- **Log4c** for the C language
- **Log4js** for the JavaScript language
- **Apache log4net**, a port to the Microsoft .NET framework

Several other ports exist, with many different logging frameworks that share many of their concepts.

Since there are many logging frameworks for the Java platform alone, there are also some wrapper logging frameworks, such as **Apache Commons Logging** or **Simple Logging Facade for Java (SLF4J)**. These are intended to allow you to use a single interface and swap out the underlying logging framework implementation if needed.

Logback is meant to be the successor of Log4j and is compatible with the ELK stack.

Log4j is the first of the Java logging frameworks, and essentially gives you the equivalent of the previous printf statements but with many more bells and whistles.

Log4j works with three primary constructs:

- Loggers
- Appenders
- Layouts

The logger is the class you use to access logging methods. There's a logging method to call for each log severity. Loggers are also hierarchical.

These concepts are easiest to explain with some example code:

```
Logger logger = Logger.getLogger("se.matangle");
```

This gets us a logger that is specific to our organization. In this way, we can make out our own logger messages from messages that originate with other organizations' code that we might be using. This becomes very useful in a Java enterprise environment, where you might have dozens of libraries that all use Log4j for their logging and you want to be able to configure different log levels for each library.

We can use several loggers at the same time for greater granularity of our logs. Different software components can have different loggers.

Here is an example from a video recording application:

```
Logger videologger = Logger.getLogger("se.matangle.video");  
logger.warn("disk is dangerously low")
```

We can use several different log levels, adding greater specificity to our logs:

```
videologger.error("video    encoding    source    running    out  
prematurely")
```

The place where the log message winds up in the end is called an **appender** in Log4j terminology. There are a number of appenders available, such as the console, files, network destinations, and logger daemons.

Layouts control how our log messages will be formatted. They allow for escape sequences with printf-like formatting.

For example, the PatternLayout class configured to use the conversion pattern `%r [%t] %-5p %c - %m%n` will create the following example output:

```
176 [main] INFO  se.matangle - User found
```

Here's what the fields in the pattern stand for:

- The first field is the number of milliseconds elapsed since the start of the program
- The second field is the thread making the log request
- The third field is the level of the log statement
- The fourth field is the name of the logger associated with the log request
- The text after the - is the message of the statement

Log4j endeavors to make the configuration of the logging external to the application. By externalizing the logging configuration, a developer can make the logging work locally by configuring logs to go to a file or to the console. Later on, when the application is deployed to a production server, the administrator can configure the log appender to be something else, such as the ELK stack we are going to discuss later. This way, the code doesn't need to be changed, and we can modify the behavior and destination of the logging at deployment time.

An application server such as WildFly offers its own configuration system that plugs in to the Log4j system.

If you don't use an application server, newer versions of Log4j support many different configuration formats. Here is an XML-style file, which will look for log4j2-test.xml in the classpath:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">

  <Appenders>

    <Console name="Console" target="SYSTEM_OUT">

      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level
%logger{36} - %msg%n"/>

    </Console>

  </Appenders>

  <Loggers>

    <Root level="error">

      <AppenderRef ref="Console"/>

    </Root>

  </Loggers>

</Configuration>
```

The ELK stack

The ELK stack consists of the following components:

- **Logstash:** This is an open source logging framework similar to Log4j. The server component of Logstash processes incoming logs.
- **Elasticsearch:** This stores all of the logs and indexes them, as the name implies, for searching.
- **Kibana:** This is the web interface for the searching and visualization of logs.

To see how it works, we will follow a log message all the way from where it originates inside the code, through the intermediary network layers, to the end destination at the network operator's screen:

1. Deep down in the code, a Java exception occurs. The application unexpectedly can't map a user ID present in an import file to the user IDs in the database. The following error is logged with log4j:

```
logger.error("cant find imported user id in database")
```

2. The log4j system is configured to log errors to a Logstash backend, as follows:

```
log4j.appender.applog=org.apache.log4j.net.SocketAppender
```

```
log4j.appender.applog.port=5000
```

```
log4j.appender.applog.remoteHost=master
```

```
log4j.appender.applog.DatePattern='.'yyyy-MM-dd
```

```
log4j.appender.applog.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.applog.layout.ConversionPattern=%d %-5p [%t] %c  
%M - %m%n
```

```
log4j.rootLogger=warn, applog
```

```
log4j.logger.nl=debug
```

3. The Logstash system is configured as follows to listen to the ports described by the log4j system. The Logstash daemon must be started separately from your application, as shown here:

```
input {  
  log4j {  
    mode => server  
    host => "0.0.0.0"  
    port => 5000  
    type => "log4j"  
  }  
}
```

```

}

output {
  stdout { codec => rubydebug }
  stdout { }

  elasticsearch {
    cluster => "elasticsearch"
  }
}

```

4. The Elasticsearch engine receives the log output and makes it searchable.
5. The administrator can open the Kibana GUI application and watch the log appear in real time and search for historical data as well.

You might think that this is a lot of complexity, and you would be correct. But it's interesting to note that Log4j can be configured to support this complex scenario as well as much simpler scenarios, so you can't really go wrong using Log4j or one of its compatible competitors.

You can start out with not using any network logging and just use plain log files. This is good enough for many situations. If and when you need the extra power of the ELK stack, you can add it later. It is useful to keep the log files around as well. If the advanced logging facilities fail for some reason, you can always resort to using the traditional Unix grep utility on the log files.

Kibana has many features to help you analyze your log data. You can graph the data and filter log data for the particular patterns that you are looking for.

To try all of this out in practice, we can use the official Kibana and Elasticsearch Docker images that are available on Docker Hub:

```
docker run -d elasticsearch &&
```

```
docker run --link some-elasticsearch:elasticsearch -d kibana
```

If all goes well, we will be able to access the Kibana user interface, which looks like this:

